CS-GY 9223 Cloud Computing and Big Data Systems

Fall 2026, Assignment 2: Flask and MongoDB To Do App on Docker, Minikube and AWS EKS

Team: Aditya Jha (aj4955) and Pragya Awasthi (pa2755)

Repository: github.com/aj4955-spec/todo-k8s Image: hub.docker.com/r/thisisadityajha/todo-flask

We containerized the starter To Do app, ran it with Docker Compose, published a multi architecture image to Docker Hub, deployed it on Minikube with three replicas behind a load balanced Service, and exercised ReplicaSets, rolling updates, health probes and Prometheus alerting to Slack. Every part of the handout is complete.

Part	What we built	Owner
1. Application	Hardened Flask app: env config, /healthz, /readyz, /version, /metrics, friendly error pages	Pragya
2. Docker	Dockerfile, Docker Compose with a named volume, multi arch image on Docker Hub	Aditya
3. Minikube	Kustomize manifests, 3 app replicas, NodePort Service, load balancing and recovery tests	Pragya
4. AWS EKS	eksctl cluster, LoadBalancer Service over the internet, MongoDB on an EBS gp3 volume	Aditya
5. ReplicaSets	Self healing, scaling up and down, declarative reset	Pragya
6. Rolling update	Zero downtime v1 to v2, two failed rollouts caught, rollback	Aditya
7. Health probes	Startup, liveness, readiness; four failure tests	Pragya
8. Alerting	Prometheus rules (Pragya), Alertmanager routing, inhibition and Slack (Aditya)	Both

Architecture

The app runs as two tiers in the todo namespace. Three Flask pods sit behind one Service and reach MongoDB through a ClusterIP Service; MongoDB keeps its data on a PersistentVolumeClaim. The same base manifests serve Minikube and EKS, and small Kustomize overlays change only the Service type and the storage class. The monitoring namespace holds Prometheus and Alertmanager, which posts to a private Slack channel.

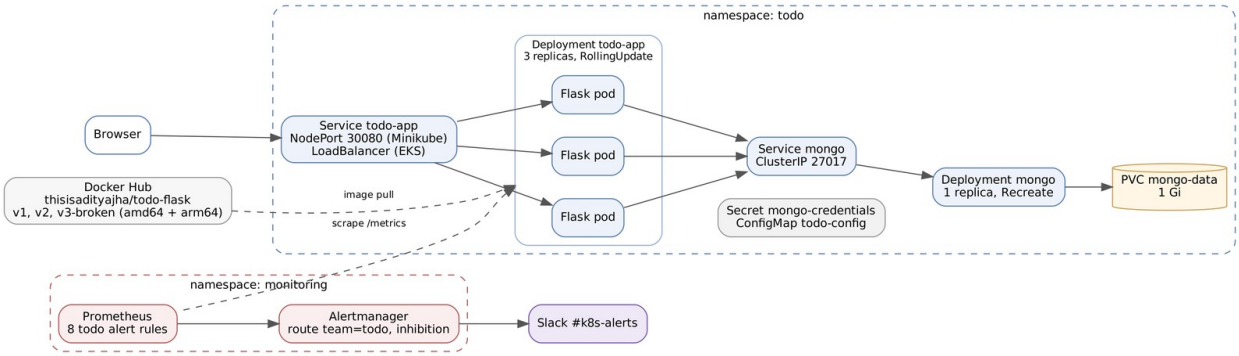


Figure 0: Request path (browser to Service to Flask pods to MongoDB), image source, and the alerting path from Prometheus through Alertmanager to Slack.

Part 1: Preparing the Application

We kept the starter app's pages and behaviour, and changed only what Kubernetes needs to run it safely. Each change below exists so that a probe, a Secret or Prometheus can work with the app.

Change	Why Kubernetes needs it
Settings read from environment variables (MONGO_URI, MONGO_DB, APP_VERSION, ENABLE_CHAOS, LOG_LEVEL)	The same image runs in Compose, Minikube and EKS. Only the ConfigMap and Secret change.
Mongo client timeouts of 2 s (server selection and connect) and 5 s (socket)	The default is 30 s, so a database outage would hang every request and every readiness check.
GET /healthz returns 200, or 500 when the flag file /tmp/unhealthy exists	Liveness and startup probe target. It never touches the database.
GET /readyz pings Mongo and returns 200 or 503	Readiness probe target. A pod that cannot reach the database leaves the load balancer.
GET /version returns the version and pod name as JSON	Shows load balancing across pods and which version is live during a rolling update.
POST /chaos/fail-liveness and POST /chaos/recover	Create or remove the flag file to test liveness on demand. They return 404 unless ENABLE_CHAOS is true.
GET /metrics via prometheus-flask-exporter	Request counts and latency by status code, scraped by Prometheus in Part 8. Probe paths are excluded so they do not drown out real traffic.
Gunicorn instead of the Flask dev server; debug=False	Production server with graceful shutdown, and no debugger exposed.
Footer showing version and pod name	A visual check in the browser of which pod served the page.

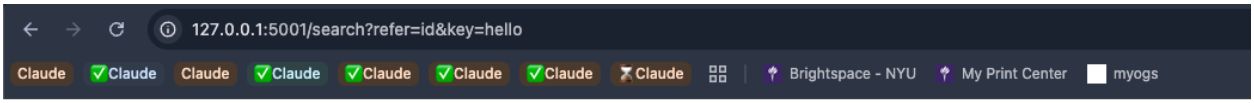
Exception handling

Every error returns a friendly HTML page with the right status code. No request ever shows a stack trace.

Situation	Response	How it is handled
Mongo unreachable	503 within about 2 s	Global handler for PyMongoError. PyMongo reconnects by itself when Mongo returns, so no restart is needed.
Task id that is not a valid ObjectId	400	parse_object_id catches InvalidId and TypeError.
Valid id that does not exist (view, toggle, edit, delete)	404	get_task_or_404, plus checks on matched_count and deleted_count.
Empty or whitespace task name	400	read_task_form trims all fields and requires a name. The HTML field is also required.
Search on a field outside the whitelist	400	Only id, name, desc, date and pr are allowed.
Search text with regex characters like (or *	Treated as plain text	re.escape before building the $regex query.
Search by an id with a bad format, or an id with no task	Page reloads with an error message	The starter app crashed with a 500 here.
Wrong HTTP method on a route	405	Dedicated handler.
Any other unexpected error	500 page	Logged with the traceback on the server side

Situation	Response	How it is handled
		only.
Chaos endpoints in production	404	Disabled unless ENABLE_CHAOS=true, and they only accept POST.

Two starter bugs were also fixed: search used the wrong id field, and the done toggle redirected to the wrong page. The edit form now keeps the task's selected priority.



Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

Figure 1.0: Before our changes, the starter app crashed with a raw 500 Internal Server Error when searching by an id that is not valid.

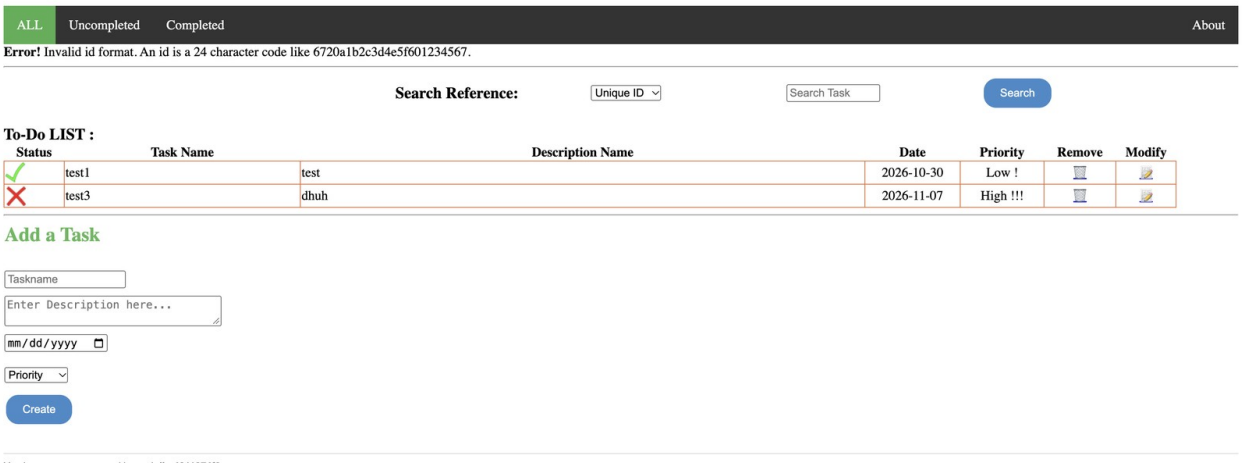


Figure 1.1: The same search on the hardened app (running in Compose) reloads the list with a clear message instead of crashing.

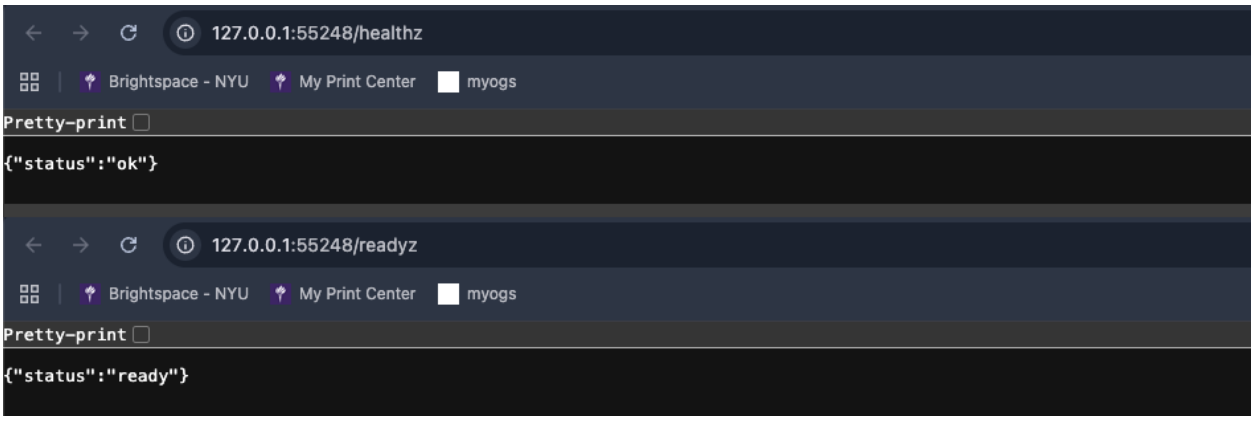


Figure 1.2: /healthz returns ok (the process is alive) and /readyz returns ready (Mongo is reachable).

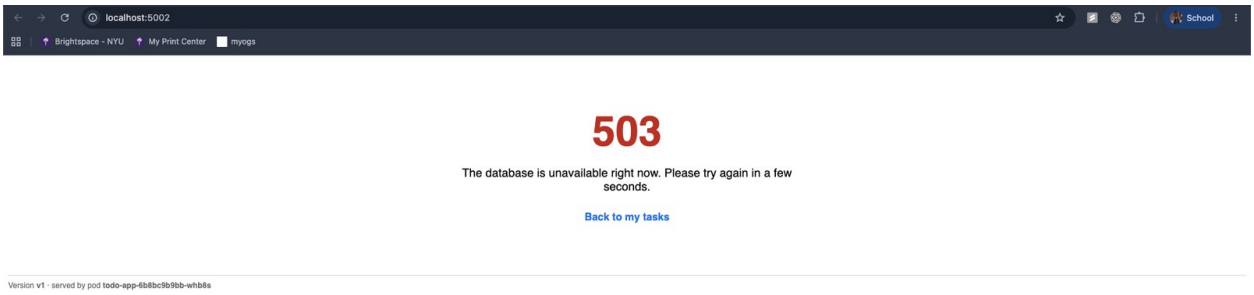


Figure 1.3: With Mongo scaled to 0, the app answers within about 2 s with a friendly 503 page instead of a crash. Taken through kubectl port-forward to one pod, because the Service itself stops routing to pods that are not ready.

Part 2: Containerizing the Application

The app runs as two containers: one for Flask and one for MongoDB. The Flask image is built from app/Dockerfile, tested locally with Docker Compose, and pushed to Docker Hub as thisisadityajha/todo-flask for both amd64 and arm64.

Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
RUN useradd --uid 10001 --create-home appuser && chown -R appuser /app
USER 10001
ARG APP_VERSION=v1
ENV APP_VERSION=${APP_VERSION}
EXPOSE 5000
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
    CMD curl -fsS http://localhost:5000/healthz || exit 1
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "1", "--threads", "4", \
     "--timeout", "30", "--graceful-timeout", "20", "--access-logfile", "-", "app:app"]
```

Choice	Reason
requirements.txt copied before the code	Dependency layers stay cached when only app code changes; a rebuild took 7 s instead of 70 s.
Non root user, UID 10001	Kubernetes runAsNonRoot works, and a compromised process has no root inside the container.
Gunicorn, 1 worker with 4 threads	Production server; scaling happens through Kubernetes replicas, and one process keeps /metrics accurate.
Exec form CMD and --graceful-timeout 20	Gunicorn is PID 1 and receives SIGTERM, so requests in flight finish during a rolling update.
APP_VERSION build argument	The same Dockerfile builds v1, v2 and v3-broken; the version shows in /version and the page footer.
HEALTHCHECK on /healthz	Docker Compose reports the container as healthy only when the app answers.
.dockerignore	Excludes .git, .env, caches and tests, so secrets and clutter never enter the image.

The image is about 185 MB unpacked and 52 MB compressed on Docker Hub. Docker Desktop shows 237 MB under disk usage because its containerd store counts the compressed download and the unpacked files together.

Docker Compose

docker-compose.yml defines the two services on a private network. Mongo stores its data in the named volume mongo-data, has a mongosh ping healthcheck, and does not publish port 27017 to

the host. The web service waits for Mongo to be healthy (depends_on: service_healthy) and reaches it at mongo:27017 with credentials from a gitignored .env file.

```
services:
  mongo:
    image: mongo:7.0
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_USER}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s
    networks: [backend]
  web:
    image: ${IMAGE:-thisisadityajha/todo-flask:v1}
    build: ./app
    ports:
      - "8080:5000"
    environment:
      MONGO_URI: mongodb://${MONGO_USER}:${MONGO_PASSWORD}@mongo:27017/?authSource=admin
      MONGO_DB: todo_db
      APP_VERSION: compose
      ENABLE_CHAOS: "true"
    depends_on:
      mongo:
        condition: service_healthy
    restart: unless-stopped
    networks: [backend]
volumes:
  mongo-data:
networks:
  backend:
```

Host port 8080 is used because macOS reserves port 5000 for AirPlay Receiver.

Compose tests

Test	What we did	Result
Start	docker compose up -d	Mongo healthy, then web started; both containers report healthy.
App works	Added, completed, deleted and searched tasks	All actions work; footer shows version compose and the container id.
Persistence	docker compose down (without -v), then up -d	Both containers recreated with a new id; tasks still present from the named volume.
Mongo down	docker compose stop mongo	/readyz 503, /healthz 200, page shows a friendly 503.

Test	What we did	Result
Recovery	docker compose start mongo	/readyz back to 200 and tasks load; web never restarted (uptime 6 minutes).
Logs	docker compose logs web	One clean ERROR line during the outage, 0 tracebacks.

```
adityajha@Mac todo-k8s % docker compose ps
curl -s localhost:8080/healthz; echo
curl -s localhost:8080/readyz; echo
curl -s localhost:8080/version; echo
NAME                IMAGE               COMMAND                  SERVICE   CREATED         STATUS                   PORTS
todo-k8s-mongo-1    mongo:7.0           "docker-entrypoint.s…"   mongo     4 minutes ago   Up 4 minutes (healthy)   27017/tcp
todo-k8s-web-1      todo-flask:compose  "gunicorn --bind 0.0…"   web       4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp
{"status":"ok"}

{"status":"ready"}

{"pod":"dba4911374f0","version":"compose"}

adityajha@Mac todo-k8s %
```

Figure 2.1: docker compose ps shows both containers healthy; only the web port is published. /healthz, /readyz and /version all answer.

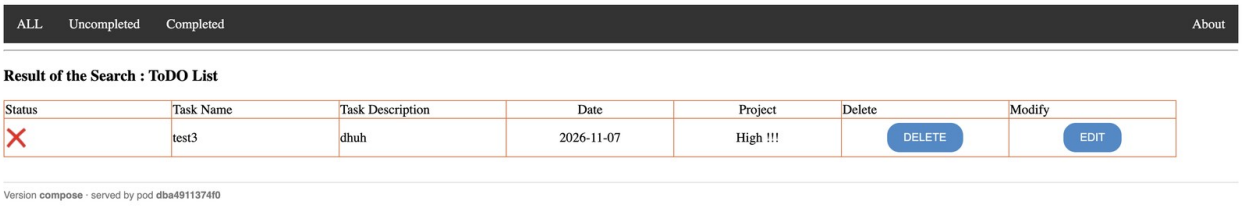


Figure 2.2: Search by task name in Compose. The footer shows version compose and the serving container id.

```
adityajha@Mac todo-k8s % docker compose down
docker compose up -d
sleep 25
docker compose ps
docker volume ls | grep mongo
[+] down 3/3
 ✔ Container todo-k8s-web-1   Removed
 ✔ Container todo-k8s-mongo-1 Removed
 ✔ Network todo-k8s_backend   Removed
[+] up 3/3
 ✔ Network todo-k8s_backend   Created
 ✔ Container todo-k8s-mongo-1 Healthy
 ✔ Container todo-k8s-web-1   Started
NAME                IMAGE               COMMAND                  SERVICE   CREATED          STATUS                    PORTS
todo-k8s-mongo-1    mongo:7.0           "docker-entrypoint.s…"   mongo     31 seconds ago   Up 30 seconds (healthy)   27017/tcp
todo-k8s-web-1      todo-flask:compose  "gunicorn --bind 0.0…"   web       31 seconds ago   Up 25 seconds (healthy)   0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp
local     todo-k8s_mongo-data
adityajha@Mac todo-k8s %
```

Figure 2.3: docker compose down and up recreate both containers and the network; the named volume todo-k8s_mongo-data survives. Web starts only after Mongo reports healthy.

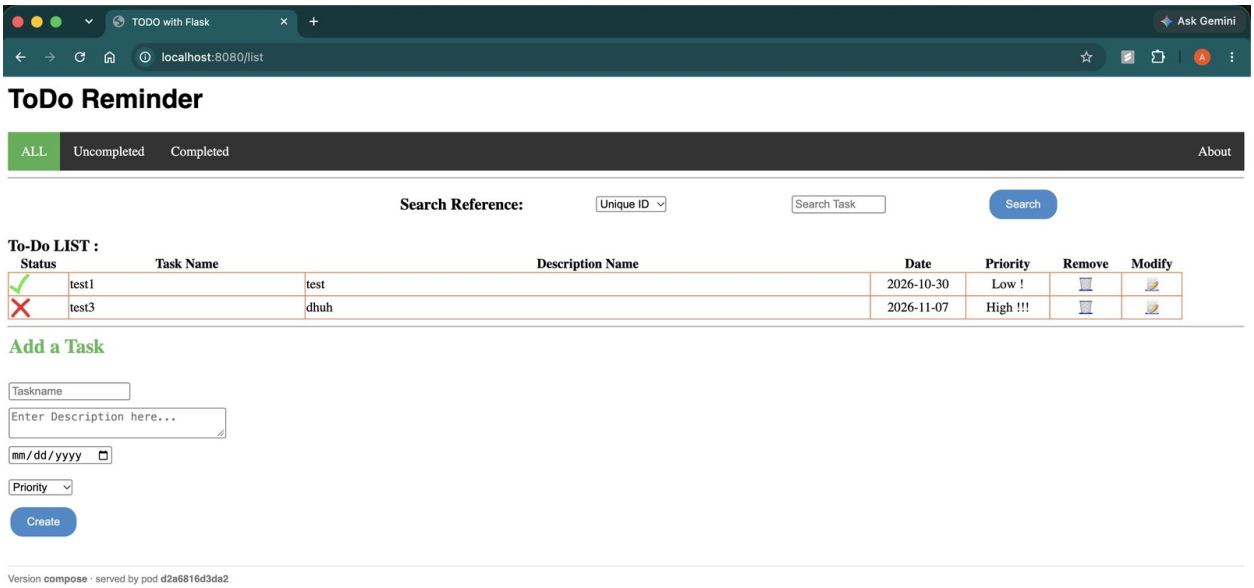


Figure 2.4: After down and up, the tasks are still there. The footer shows a new container id, so a brand new container is reading the same data.

```
adityajha@Adityas-MacBook-Pro todo-k8s % docker compose stop mongo && sleep 3
curl -s -o /dev/null -w "readyz %{http_code}\n" localhost:8080/readyz
curl -s -o /dev/null -w "healthz %{http_code}\n" localhost:8080/healthz
docker compose start mongo
[+] stop 1/1
 ✔ Container todo-k8s-mongo-1 Stopped                                    0.1s
readyz 503
healthz 200
[+] start 1/1
 ✔ Container todo-k8s-mongo-1 Started                                    0.1s
adityajha@Adityas-MacBook-Pro todo-k8s %
```

Figure 2.5: With Mongo stopped, /readyz returns 503 (cannot serve users) while /healthz stays 200 (process alive). In Kubernetes this removes the pod from the Service without restarting it.

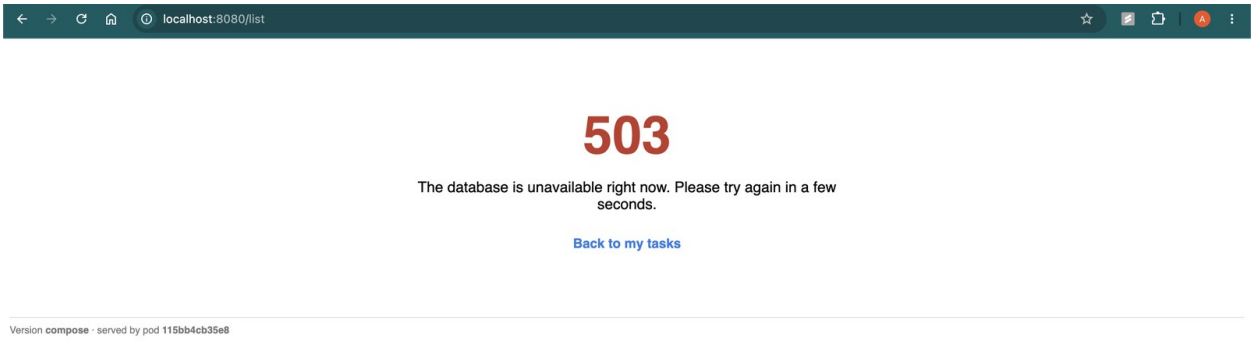


Figure 2.6: The user sees a friendly 503 page with a link back, not a stack trace.

```
adityajha@Mac todo-k8s % docker compose start mongo
sleep 20
curl -s -o /dev/null -w "readyz %{http_code}\n" localhost:8080/readyz
docker compose ps
[+] start 1/1
 ✔ Container todo-k8s-mongo-1 Started                                    0.1s
readyz 200
NAME            IMAGE               COMMAND                  SERVICE   CREATED         STATUS                   PORTS
todo-k8s-mongo-1   mongo:7.0           "docker-entrypoint.s…"   mongo     6 minutes ago   Up 20 seconds (healthy)   27017/tcp
todo-k8s-web-1     todo-flask:compose   "gunicorn --bind 0.0…"   web       6 minutes ago   Up 6 minutes (healthy)    0.0.0.0:8080->5000/tcp, [::]:8080->5000/tcp
adityajha@Mac todo-k8s %
```

Figure 2.7: After docker compose start mongo, /readyz is 200 again and web has been up for 6 minutes: the app reconnected without a restart.

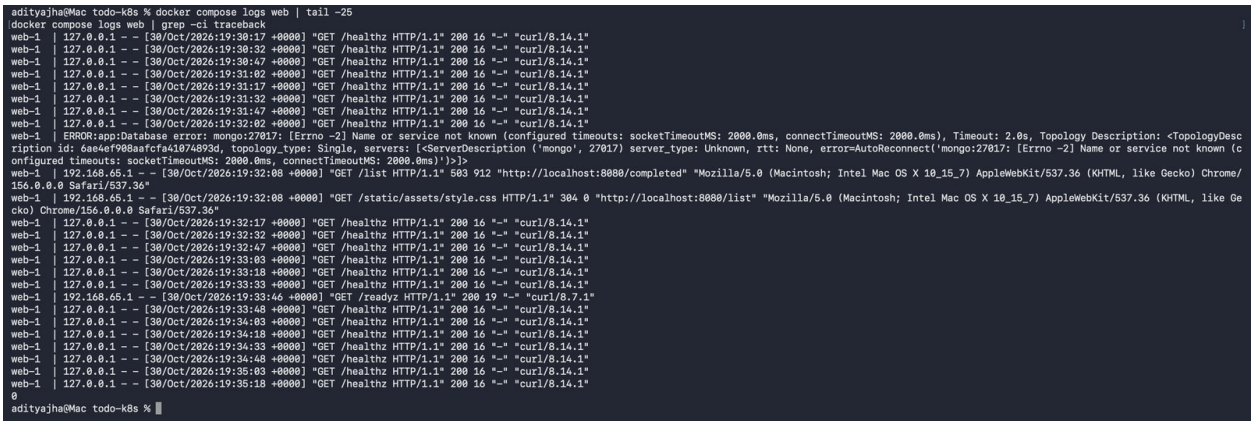


Figure 2.8: Web logs show access lines, one ERROR line while Mongo was down, the 503 response, and 0 tracebacks.

Docker Hub

Images are built with docker buildx for linux/amd64 (EKS nodes) and linux/arm64 (Apple Silicon Minikube). Tags are never overwritten; a fix would get a new tag such as v1.1.

Tag	Contents
v0-starter	Unmodified starter app, used only to test the multi arch push pipeline.
v1	Hardened app (Part 1). First release on Minikube.
v2	Same code with APP_VERSION=v2, visible in /version and the footer. Target of the rolling update.
v3-broken	app.py imports a module that does not exist, so gunicorn exits with code 3. Used for the failed rollout and probe tests.

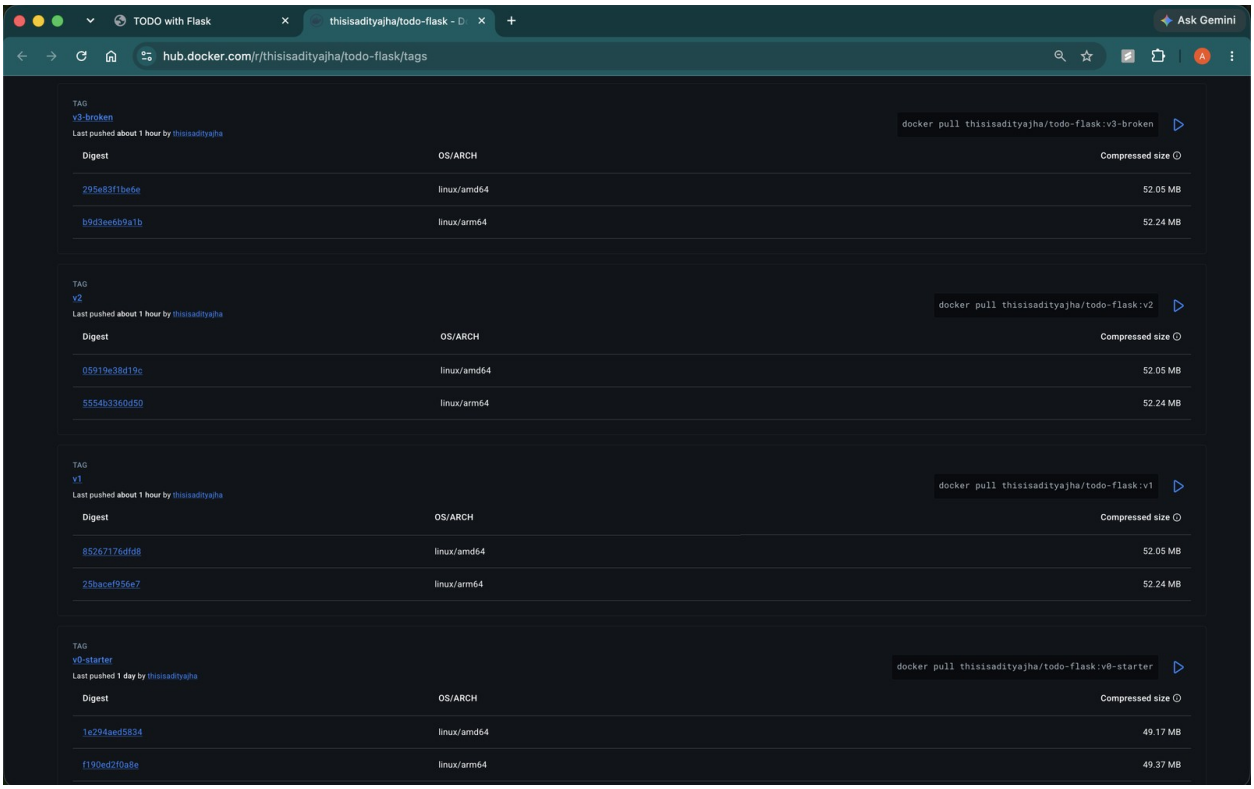


Figure 2.9: Docker Hub repository thisisadityajha/todo-flask. Every tag has an amd64 and an arm64 image of about 52 MB compressed.

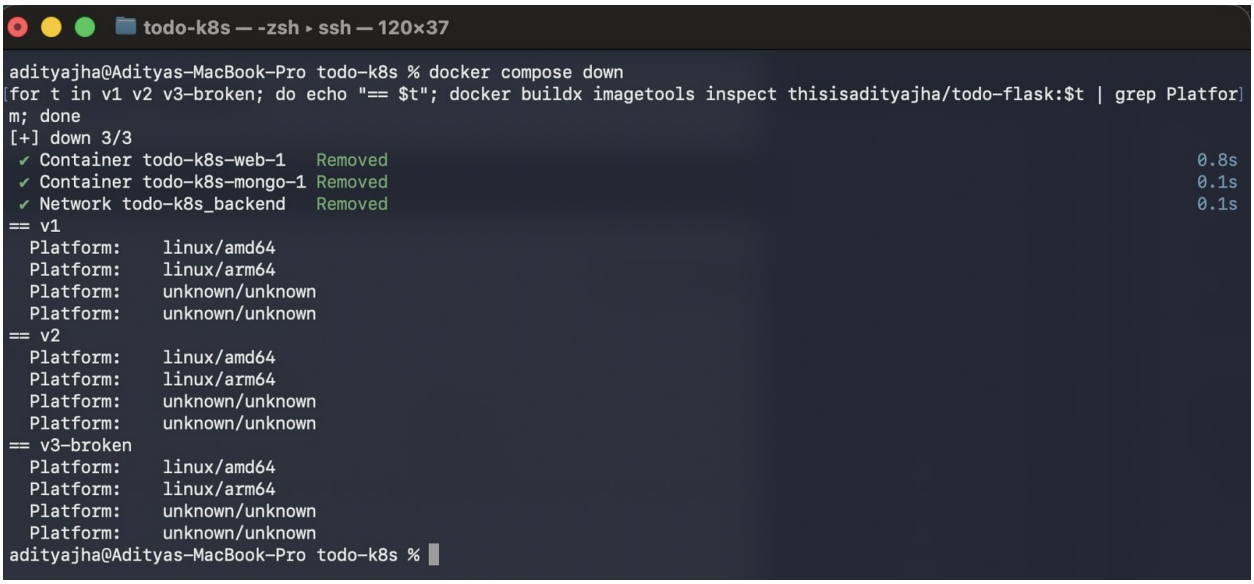


Figure 2.10: docker buildx imagetools inspect lists linux/amd64 and linux/arm64 for each tag. The unknown/unknown entries are build attestations, not images.

Part 3: Deploying on Minikube

The app runs on Minikube as two tiers in the todo namespace: three Flask pods behind one Service, and one MongoDB pod whose data lives on a PersistentVolumeClaim. All manifests are in k8s/base, and k8s/overlays/minikube adapts them for Minikube with Kustomize. The handout asks for two pods; we read that as two kinds of pods, with the app tier replicated.

File	Object	Key choices
namespace.yaml	Namespace todo	Keeps the app apart from the monitoring stack and makes cleanup one command.
app-configmap.yaml	ConfigMap todo-config	Non secret settings: MONGO_DB=todo_db, ENABLE_CHAOS=true, LOG_LEVEL=INFO.
mongo-secret.example.yaml	Secret mongo-credentials (template only)	The real Secret is created with kubectl create secret, so no password is ever committed to Git.
mongo-pvc.yaml	PVC mongo-data, 1 Gi, ReadWriteOnce	Data outlives the Mongo pod. Minikube's default StorageClass provisions it.
mongo-deployment.yaml	Deployment mongo, 1 replica	Recreate strategy, so two Mongo pods never mount the same volume at once. Credentials come from the Secret. Probes run a mongosh ping.
mongo-service.yaml	Service mongo, ClusterIP, port 27017	Database is reachable only inside the cluster, at the stable DNS name mongo.
app-deployment.yaml	Deployment todo-app, 3 replicas	Probes (Part 7), RollingUpdate with maxSurge 1 and maxUnavailable 0, progressDeadlineSeconds 120, runs as UID 10001, requests 100m CPU and 128 Mi, limits 500m and 256 Mi, preStop sleep 5 s.
app-service.yaml	Service todo-app, port 80 to container port 5000	ClusterIP in the base. The Minikube overlay patches it to NodePort 30080; the EKS overlay makes it a LoadBalancer.
kustomization.yaml	Kustomize base	Sets the namespace and the image thisisadityajha/todo-flask with the current release tag (v1 at first deploy, v2 after Part 6).

The app builds its connection string inside the pod: MONGO_URI is mongodb://$(MONGO_USER):$(MONGO_PASSWORD)@mongo:27017/?authSource=admin. Kubernetes fills in both variables from the Secret, because they are listed before MONGO_URI in the container spec.

```
minikube start --driver=docker --cpus=4 --memory=8192
kubectl apply -f k8s/base/namespace.yaml
kubectl create secret generic mongo-credentials -n todo \
  --from-literal=MONGO_USER=<user> --from-literal=MONGO_PASSWORD=<password>
kubectl apply -k k8s/overlays/minikube
kubectl get all,pvc -n todo
minikube service todo-app -n todo --url
```

Test	What we did	What happened
Deployment	kubectl apply -k k8s/overlays/minikube	3 app pods and 1 Mongo pod Running, PVC Bound. App pods became ready only after Mongo was ready.
Load balancing	9 requests to /version through the Service URL	Answers came from all 3 pods.

Test	What we did	What happened
Pod recovery	Deleted app pod 4r9jz	The replacement trtwr was Running and ready 4 s after it was created.
Data persistence	Added tasks, then deleted the Mongo pod twice	Each new Mongo pod was ready in about 7 s and every task was still there, because the data lives on the PVC.

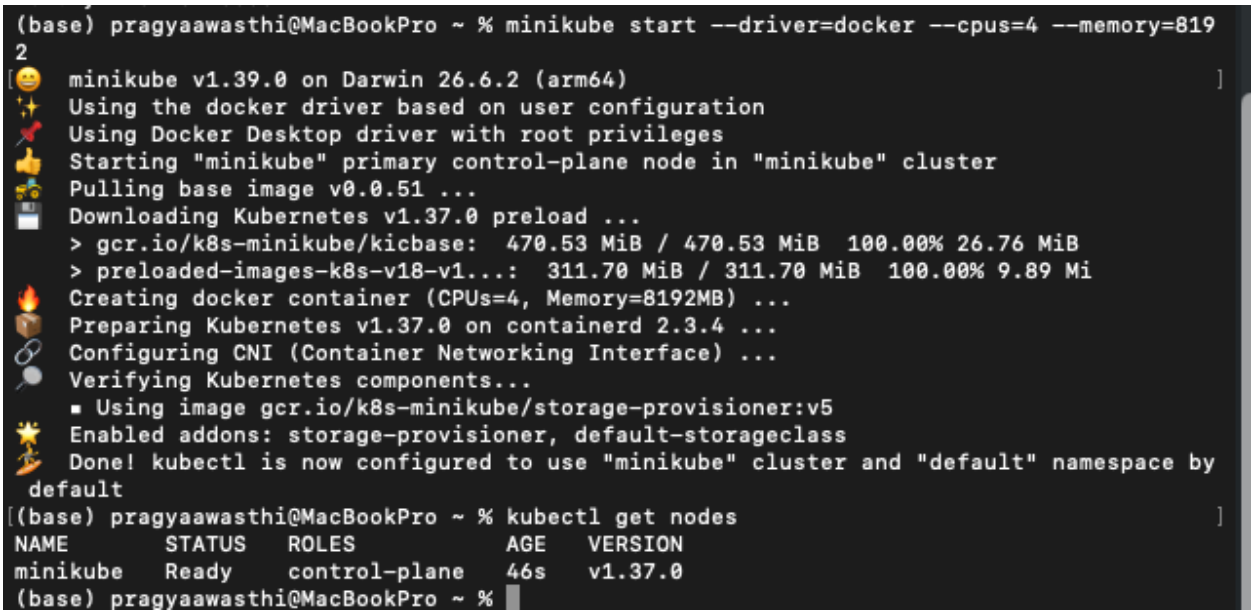


Figure 3.1: Minikube v1.39.0 starting Kubernetes v1.37.0 on the docker driver with 4 CPUs and 8192 MB, and the single node Ready.

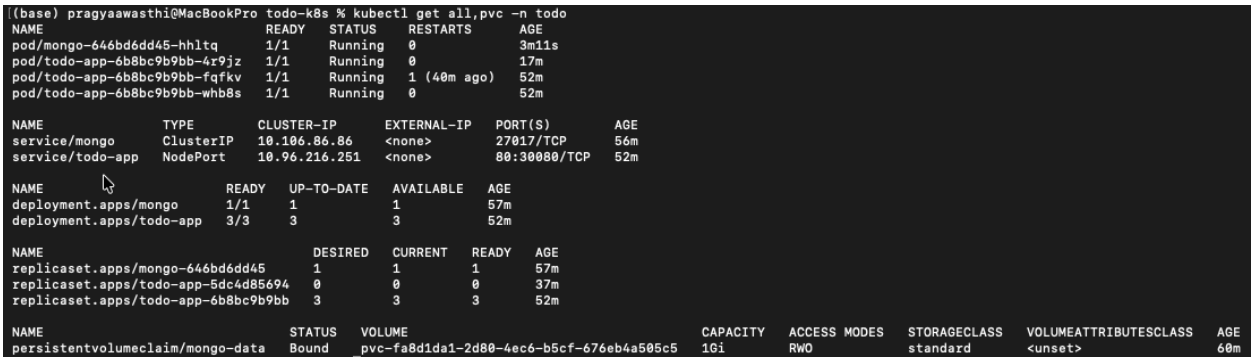


Figure 3.2: Everything in the todo namespace: 4 pods at 1/1, the ClusterIP mongo Service and the NodePort todo-app Service (80:30080), both Deployments, and the PVC mongo-data Bound at 1 Gi. The ReplicaSet at 0 replicas is the broken image from Part 7, kept as rollout history.

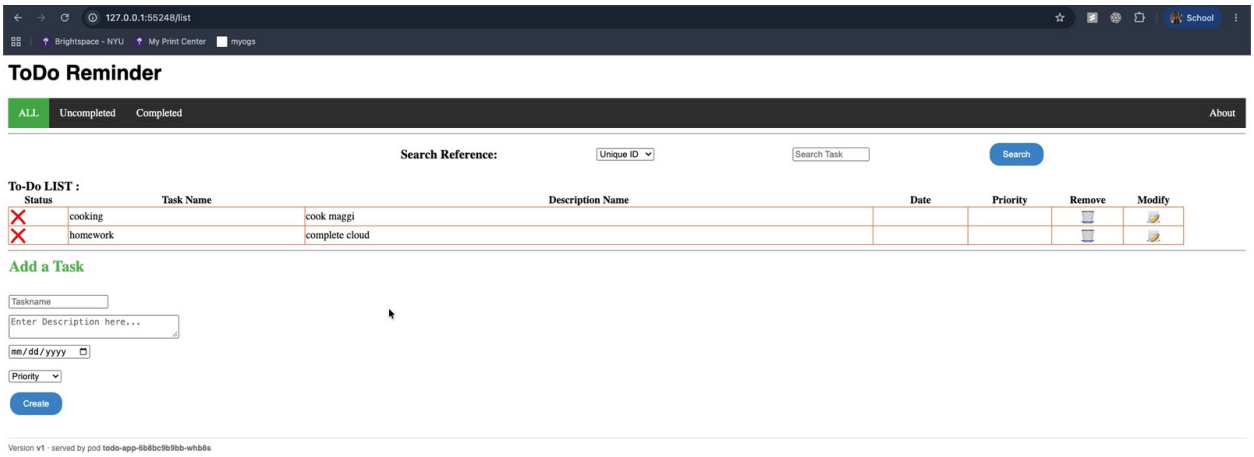


Figure 3.3: The app through the Minikube Service URL, served by pod whb8s at version v1. The tasks survived every Mongo pod deletion and every scale to 0 during testing, because the data lives on the PVC.

```
(base) pragyaawasthi@MacBookPro todo-k8s % for i in $(seq 1 9); do curl -s http://127.0.0.1:55248/version; echo; done
{"pod":"todo-app-6b8bc9b9bb-whb8s","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-whb8s","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-fqfkv","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-4r9jz","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-whb8s","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-fqfkv","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-4r9jz","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-4r9jz","version":"v1"}

{"pod":"todo-app-6b8bc9b9bb-4r9jz","version":"v1"}

(base) pragyaawasthi@MacBookPro todo-k8s %
```

Figure 3.4: Nine requests to /version through one Service URL were answered by all 3 pods (whb8s, fqfkv, 4r9jz), so the Service spreads traffic across the replicas.

```
(base) pragyaawasthi@MacBookPro todo-k8s % kubectl delete pod -n todo todo-app-6b8bc9b9bb-4r9jz --wait=false
kubectl get pods -n todo -l app=todo-app -w
pod "todo-app-6b8bc9b9bb-4r9jz" deleted from todo namespace
NAME                        READY   STATUS              RESTARTS      AGE
todo-app-6b8bc9b9bb-4r9jz   1/1     Terminating         0             20m
todo-app-6b8bc9b9bb-fqfkv   1/1     Running             1 (44m ago)   56m
todo-app-6b8bc9b9bb-trtwr   0/1     ContainerCreating   0             0s
todo-app-6b8bc9b9bb-whb8s   1/1     Running             0             56m
todo-app-6b8bc9b9bb-trtwr   0/1     ContainerCreating   0             0s
todo-app-6b8bc9b9bb-trtwr   0/1     Running             0             1s
todo-app-6b8bc9b9bb-trtwr   0/1     Running             0             3s
todo-app-6b8bc9b9bb-trtwr   0/1     Running             0             3s
todo-app-6b8bc9b9bb-trtwr   1/1     Running             0             4s
todo-app-6b8bc9b9bb-4r9jz   0/1     Completed           0             20m
todo-app-6b8bc9b9bb-4r9jz   0/1     Completed           0             20m
todo-app-6b8bc9b9bb-4r9jz   0/1     Completed           0             20m
```

Figure 3.5: Pod 4r9jz is deleted and goes Terminating. The ReplicaSet immediately creates trtwr, which goes from ContainerCreating to Running 0/1, then passes readiness and is 1/1 just 4 s later.

Part 4: Deploying on AWS EKS

The same manifests run on a two node EKS cluster in us-east-1: the app is reachable from the internet through an AWS load balancer, traffic is spread across all three pods, and MongoDB data lives on an EBS gp3 volume that survived a pod deletion.

Cluster

eksctl created cluster todo-eks (Kubernetes 1.34) from eks/cluster.yaml: a managed node group of two m7i-flex.large nodes (2 vCPU, 8 GB) in two availability zones, public subnets with no NAT gateway to save cost, OIDC enabled, and the EBS CSI driver add-on with its own IAM role so Kubernetes can create EBS volumes.

```
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig
metadata:
  name: todo-eks
  region: us-east-1
  tags: {project: cloud-a2}
vpc:
  nat:
    gateway: Disable
iam:
  withOIDC: true
addons:
  - name: vpc-cni
  - name: coredns
  - name: kube-proxy
  - name: aws-ebs-csi-driver
    wellKnownPolicies:
      ebsCSIController: true
managedNodeGroups:
  - name: ng-2
    instanceType: m7i-flex.large
    desiredCapacity: 2
    minSize: 2
    maxSize: 3
    volumeSize: 20
    labels: {role: worker}

eksctl create cluster -f eks/cluster.yaml --dry-run      # validate first
eksctl create cluster -f eks/cluster.yaml
aws eks update-kubeconfig --name todo-eks --region us-east-1
kubectl config current-context                           # must be the EKS context
eksctl create addon -f eks/addon-ebs.yaml                # EBS CSI driver
kubectl get nodes -o wide -L topology.kubernetes.io/zone
```

Issue: AWS free plan blocks t3.medium

The first node group (ng-1, t3.medium) never produced a node. Its Auto Scaling group logged "The specified instance type is not eligible for Free Tier": the account is on the AWS free plan, which only launches free tier eligible types. A dry run of RunInstances does not check this, so it only showed up on the real launch. We listed the allowed types with aws ec2 describe-instance-types --filters

Name=free-tier-eligible,Values=true, created node group ng-2 with m7i-flex.large (more memory than t3.medium), installed the EBS CSI add-on that eksctl had not reached, and deleted ng-1.

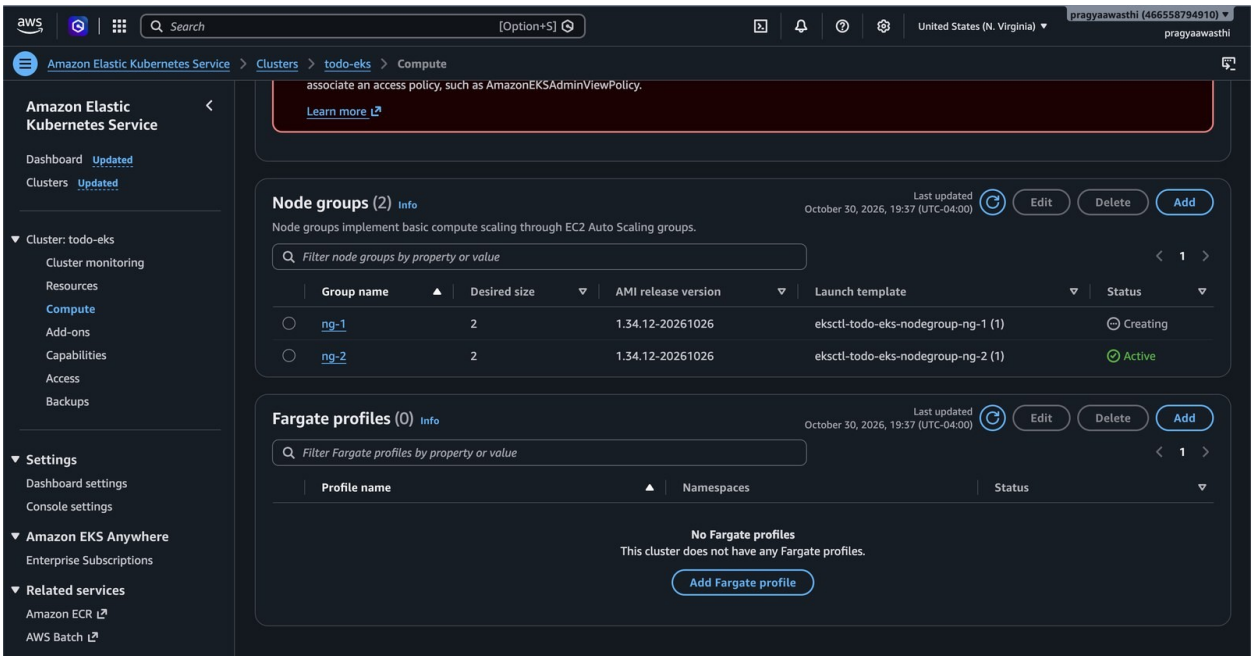


Figure 4.1: Node groups in the EKS console. ng-1 (t3.medium) is stuck in Creating because the free plan refused the instance type; ng-2 (m7i-flex.large) is Active with 2 nodes.

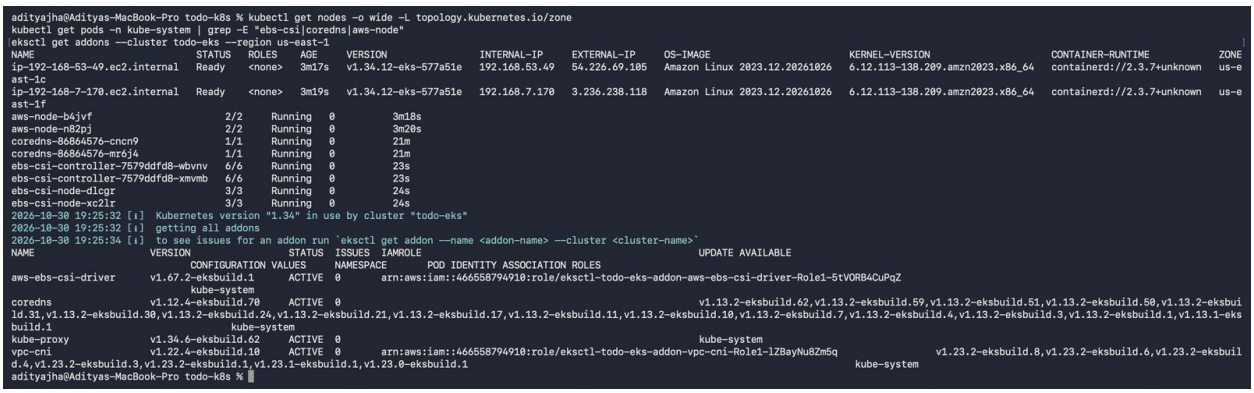


Figure 4.2: Two Ready nodes on Kubernetes 1.34 in us-east-1c and us-east-1f, the EBS CSI controller and node pods Running, and every add-on ACTIVE, aws-ebs-csi-driver with its own IAM role.

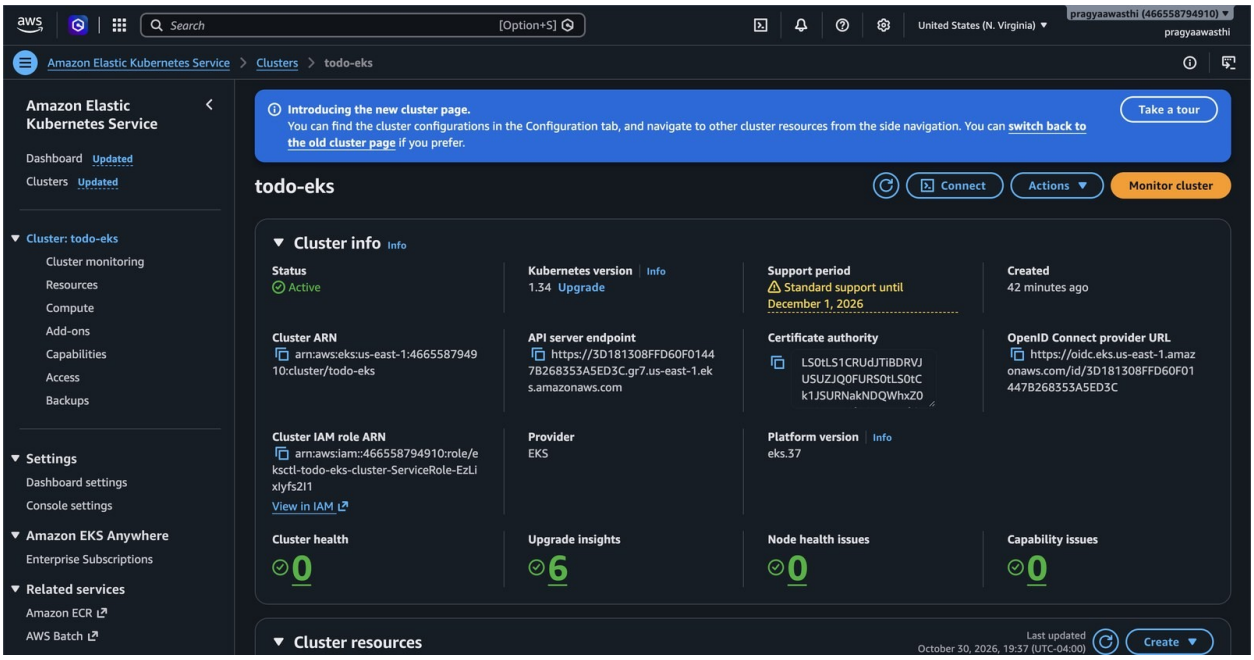


Figure 4.3: Cluster todo-eks in the EKS console: Active, Kubernetes 1.34, OIDC provider configured, 0 node health issues.

EKS overlay

k8s/overlays/eks reuses k8s/base unchanged and adds two things: a gp3 StorageClass for the EBS CSI driver, and patches that make the app Service a LoadBalancer and point the Mongo PVC at gp3. WaitForFirstConsumer creates the volume only after Mongo is scheduled, in that node's zone, because an EBS volume cannot cross availability zones.

```
# k8s/overlays/eks/storageclass-gp3.yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Delete
allowVolumeExpansion: true
parameters:
  type: gp3
  fsType: ext4

# k8s/overlays/eks/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
  - storageclass-gp3.yaml
patches:
  - target: {kind: Service, name: todo-app}
    patch: |-
      - op: replace
        path: /spec/type
```

```
        value: LoadBalancer
  - target: {kind: PersistentVolumeClaim, name: mongo-data}
    patch: |-
      - op: add
        path: /spec/storageClassName
        value: gp3
```

Deployment

```
kubectl apply -f k8s/base/namespace.yaml
kubectl create secret generic mongo-credentials -n todo \
  --from-literal=MONGO_USER=<user> --from-literal=MONGO_PASSWORD=<password>
kubectl apply -k k8s/overlays/eks
kubectl -n todo get pods -w
kubectl -n todo get all,pvc
kubectl get pv
```

Mongo was 1/1 after 29 s, including creating and attaching the EBS volume, and the three app pods passed readiness 2 s later, as soon as they could reach the database.

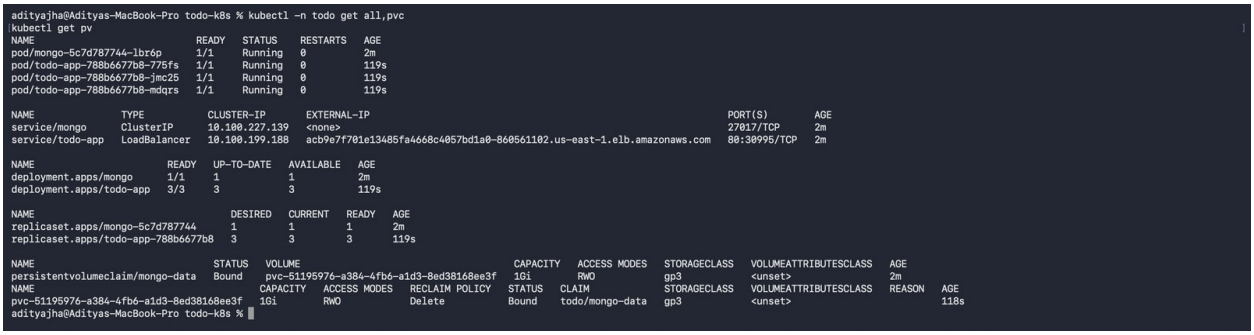


Figure 4.4: Everything in the todo namespace on EKS. The todo-app Service is a LoadBalancer with a public ELB hostname; both Deployments are fully available; PVC mongo-data is Bound to a 1 Gi gp3 volume.

Testing from outside the cluster

```
ELB=$(kubectl -n todo get svc todo-app -o
jsonpath='{.status.loadBalancer.ingress[0].hostname}')
nslookup $ELB
for i in $(seq 1 12); do curl -s -m 5 http://$ELB/version; echo; done
```

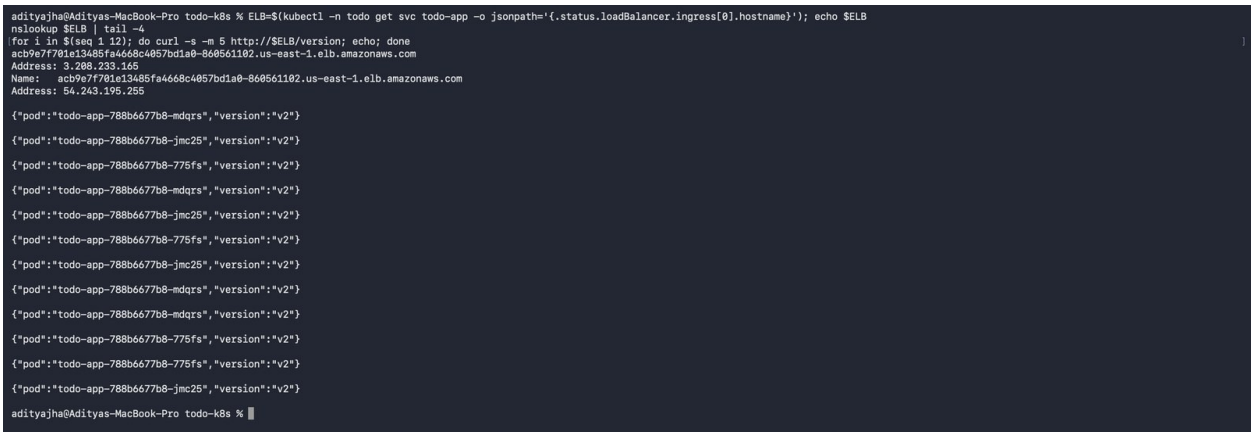


Figure 4.5: The ELB name resolves to two public IPs, one per zone. Twelve requests were answered 4 times each by the three pods, all running v2.

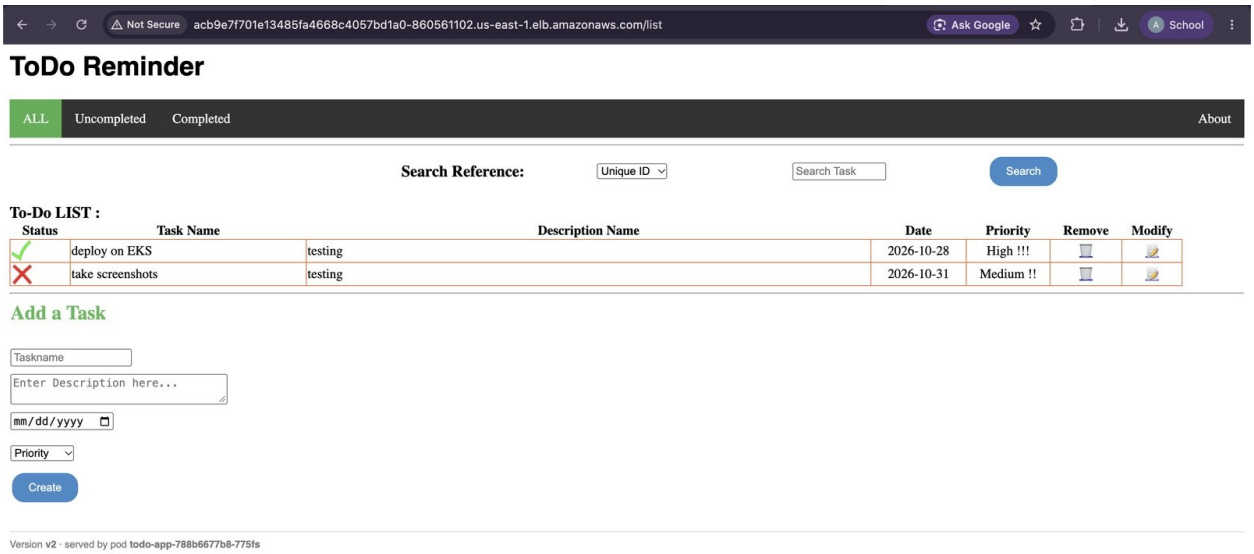


Figure 4.6: The app opened through the ELB address over the internet. Tasks are stored in MongoDB on EBS; the footer shows v2 served by EKS pod 775fs.

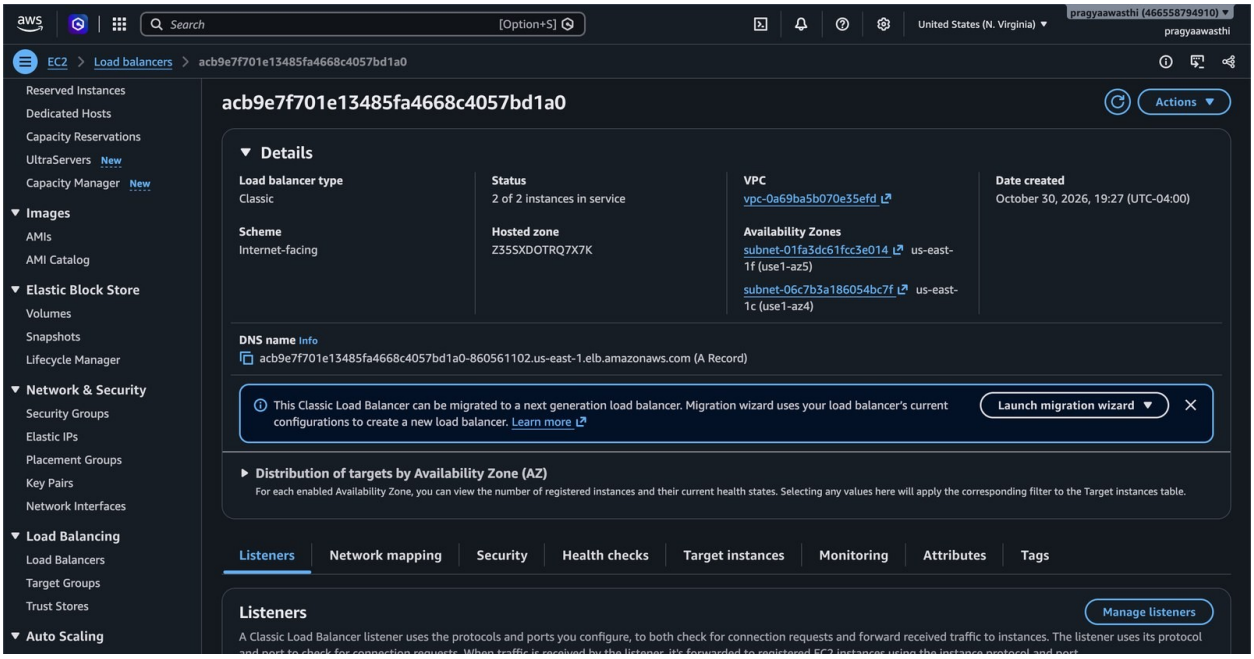


Figure 4.7: The load balancer in the EC2 console: internet facing, 2 of 2 instances in service across us-east-1f and us-east-1c, with the same DNS name. It is a Classic Load Balancer because that is what a LoadBalancer Service creates on EKS without the AWS Load Balancer Controller.

Data persistence on EBS

Step	Result
Before	Mongo pod lbr6p on node ip-192-168-53-49 (us-east-1c)
kubectl -n todo delete pod -l app=mongo	Replacement r97cj created on the same node, 1/1 after 7 s
kubectl get pv	Same PV, backed by EBS volume vol-05272767aa8eedc04 in us-east-1c, still bound to mongo-data

Step	Result
Browser refresh	Both tasks still present, now served by a different pod (mdqrs)

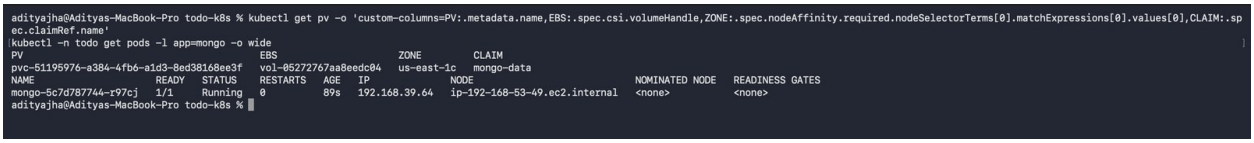


Figure 4.8: After the deletion, the PV is still backed by EBS volume vol-05272767aa8eedc04 in us-east-1c, and the new Mongo pod r97cj runs on a node in that zone.

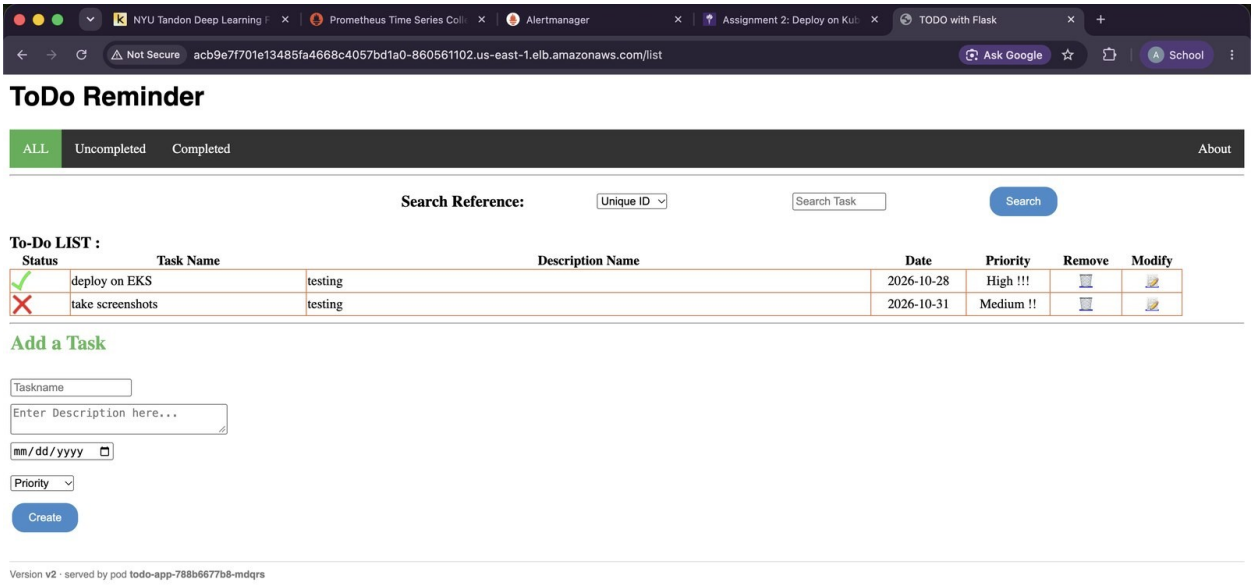


Figure 4.9: The tasks survived the Mongo pod deletion, because the data lives on the EBS volume, not in the pod.

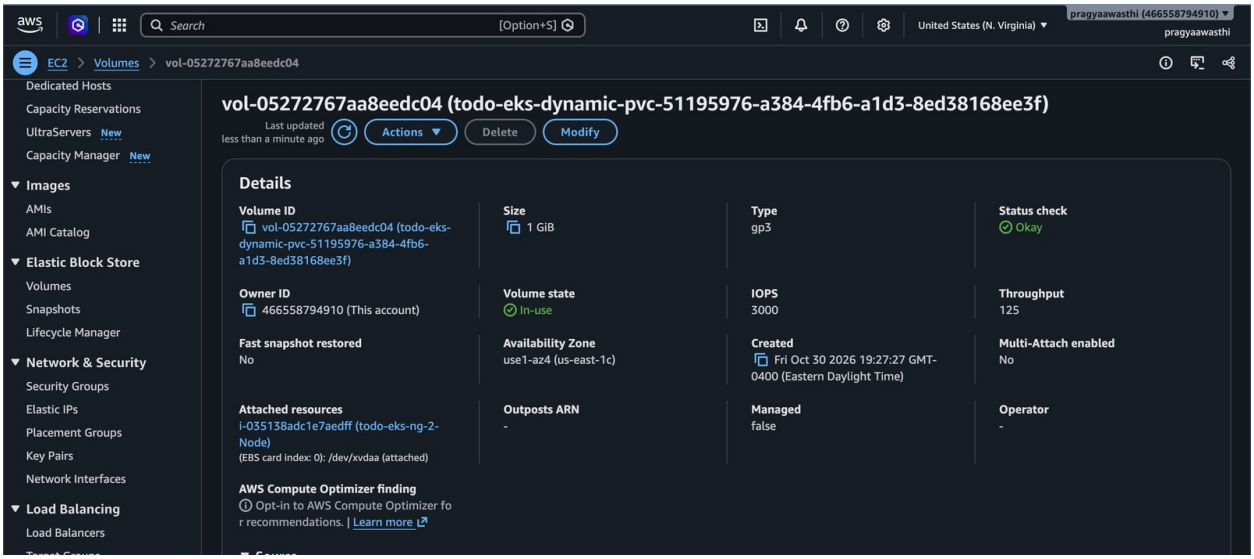


Figure 4.10: The EBS volume in the console: named after the Kubernetes PV, 1 GiB gp3, In-use in us-east-1c, attached to an ng-2 node.

Cost and teardown

Running cost is roughly 0.30 USD per hour: about 0.10 for the control plane, about 0.20 for two m7i-flex.large nodes, plus a few cents for the load balancer and the 1 GiB volume. A 25 USD budget with alerts was set before creating anything. After the screenshots, the cluster was deleted the same evening; eksctl reported all cluster resources were deleted, and the EBS volume and load balancer were confirmed gone. Teardown order matters, so the load balancer and EBS volume are removed by Kubernetes before the cluster goes:

```
# 1. Delete the Kubernetes objects: removes the ELB and (reclaimPolicy Delete) the EBS volume
kubectl delete -k k8s/overlays/eks
# 2. Confirm the volume is gone
aws ec2 describe-volumes --volume-ids vol-05272767aa8eedc04
# 3. Delete the cluster
eksctl delete cluster -f eks/cluster.yaml --wait
```

Part 5: Deployments and ReplicaSets

We used a Deployment, which creates and owns a ReplicaSet, instead of a bare ReplicationController. ReplicationController is the legacy API, and the rolling updates in Part 6 need a Deployment anyway. The ReplicaSet keeps exactly the number of pods the Deployment asks for, and replaces any that disappear.

The ReplicaSet's name todo-app-6b8bc9b9bb ends in a pod-template-hash. That hash is how the Deployment tells the ReplicaSet of one version from another during a rolling update, and every pod it owns carries the same hash in its name and labels.

Test	Command	What happened
Inspect	kubectl get deployment,replicaset,pods -n todo -l app=todo-app	Deployment 3/3, ReplicaSet desired 3, current 3, ready 3.
Ownership	kubectl describe replicaset -n todo -l app=todo-app	Controlled By: Deployment/todo-app. Events list each pod it created, including the replacement for the deleted pod.
Self healing	kubectl delete pod -n todo <pod>	The ReplicaSet created a new pod at once, back to 3 ready in a few seconds.
Scale up	kubectl scale deployment todo-app -n todo --replicas=5	2 new pods Running within 10 s.
Scale down	kubectl scale deployment todo-app -n todo --replicas=2	3 pods removed within 10 s. The two oldest pods were kept.
Back to the file	kubectl apply -k k8s/overlays/minikube	Replicas returned to 3, the count in the YAML.

The last test matters: kubectl scale is imperative, so the next kubectl apply resets it to what Git says. We treat the YAML as the source of truth, and change replicas there for any lasting change.



Figure 5.1: The Deployment todo-app at 3/3 and its ReplicaSet 6b8bc9b9bb at 3 desired, 3 current, 3 ready. Every pod name carries the same hash. The second ReplicaSet at 0 is the broken version from Part 7, kept for rollback history.

```
Name:           todo-app-6b8bc9b9bb
Namespace:      todo
Selector:       app=todo-app,pod-template-hash=6b8bc9b9bb
Labels:         app=todo-app
                part-of=todo
                pod-template-hash=6b8bc9b9bb
Annotations:    deployment.kubernetes.io/desired-replicas: 3
                deployment.kubernetes.io/max-replicas: 4
                deployment.kubernetes.io/revision: 3
                deployment.kubernetes.io/revision-history: 1
                kubernetes.io/change-cause: initial deploy
Controlled By:  Deployment/todo-app
  Normal  SuccessfulCreate  48m   replicaset-controller  Created pod: todo-app-6b8bc9b9bb-txgxb
  Normal  SuccessfulCreate  48m   replicaset-controller  Created pod: todo-app-6b8bc9b9bb-jqsvv
  Normal  SuccessfulDelete  48m   replicaset-controller  Deleted pod: todo-app-6b8bc9b9bb-t4shq
  Normal  SuccessfulDelete  48m   replicaset-controller  Deleted pod: todo-app-6b8bc9b9bb-jqsvv
  Normal  SuccessfulDelete  48m   replicaset-controller  Deleted pod: todo-app-6b8bc9b9bb-txgxb
  Normal  SuccessfulCreate  47m   replicaset-controller  Created pod: todo-app-6b8bc9b9bb-9zcbn
  Normal  SuccessfulCreate  22m   replicaset-controller  Created pod: todo-app-6b8bc9b9bb-4r9jz
  Normal  SuccessfulCreate  89s   replicaset-controller  Created pod: todo-app-6b8bc9b9bb-trtwr
```

Figure 5.2: The ReplicaSet's selector includes pod-template-hash, and it is Controlled By: Deployment/todo-app. Its events record pods created and deleted by every scale and every replacement.

```
deployment.apps/todo-app scaled
NAME                        READY   STATUS    RESTARTS      AGE
todo-app-6b8bc9b9bb-2l4gb   1/1     Running   0             10s
todo-app-6b8bc9b9bb-fqfkv   1/1     Running   1 (46m ago)   59m
todo-app-6b8bc9b9bb-trtwr   1/1     Running   0             2m31s
todo-app-6b8bc9b9bb-whb8s   1/1     Running   0             59m
todo-app-6b8bc9b9bb-wj9nt   1/1     Running   0             10s
deployment.apps/todo-app scaled
NAME                        READY   STATUS    RESTARTS      AGE
todo-app-6b8bc9b9bb-fqfkv   1/1     Running   1 (46m ago)   59m
todo-app-6b8bc9b9bb-whb8s   1/1     Running   0             59m
```

Figure 5.3: Scaled to 5, two new pods (2l4gb, wj9nt) were Running and ready within 10 s. Scaled to 2, the ReplicaSet removed the three newest pods and kept the two oldest.

```
(base) pragyaawasthi@MacBookPro todo-k8s % git checkout pragya/k8s-manifests
kubectl apply -k k8s/overlays/minikube
sleep 10
kubectl get pods -n todo -l app=todo-app
Switched to branch 'pragya/k8s-manifests'
Your branch is up to date with 'origin/pragya/k8s-manifests'.
namespace/todo unchanged
configmap/todo-config unchanged
service/mongo unchanged
service/todo-app unchanged
persistentvolumeclaim/mongo-data unchanged
deployment.apps/mongo unchanged
deployment.apps/todo-app configured
NAME                        READY   STATUS    RESTARTS      AGE
todo-app-6b8bc9b9bb-dxt8g   1/1     Running   0             10s
todo-app-6b8bc9b9bb-fqfkv   1/1     Running   1 (48m ago)   60m
todo-app-6b8bc9b9bb-whb8s   1/1     Running   0             60m
```

Figure 5.4: kubectl apply -k reports only deployment.apps/todo-app configured, since every other object already matches the files. The replica count goes back to the 3 in the YAML, with one new pod (dxt8g).

Part 6: Rolling Update Strategy

The update from v1 to v2 ran with zero failed requests: a request loop through the Service returned 200 on every call while pods were replaced one at a time. Two deliberately bad releases were stopped before they reached users and were rolled back with one command.

Strategy

```
spec:
  replicas: 3
  progressDeadlineSeconds: 120
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1        # at most 4 pods during the update
      maxUnavailable: 0  # never fewer than 3 ready pods
```

maxUnavailable 0 means Kubernetes removes an old pod only after a new one passes its readiness probe, so a release that never becomes ready can never take capacity away. maxSurge 1 keeps the extra resource cost to one pod. The 120 s progress deadline marks a stuck rollout as failed instead of waiting forever. Graceful shutdown (preStop sleep 5 s, gunicorn graceful timeout 20 s) lets requests in flight finish.

Update from v1 to v2

```
while true; do echo "$(date +%T) $(curl -s -m 2 -w ' %{http_code}' $URL/version)"; sleep 0.5;
done
kubectl -n todo get rs -w
kubectl -n todo set image deploy/todo-app todo-app=thisisadityajha/todo-flask:v2
kubectl -n todo annotate deploy/todo-app kubernetes.io/change-cause="Rolling update v1 to v2"
--overwrite
kubectl -n todo rollout status deploy/todo-app
kubectl -n todo rollout history deploy/todo-app
```

After the rollout, newTag in k8s/base/kustomization.yaml was changed to v2 and committed, so a later kubectl apply -k does not silently roll back to v1. kubectl diff confirmed the files and the cluster match.



Figure 6.1: Traffic loop during the update. The first v2 pod (lxgp4) starts serving at 16:23:31 while v1 pods still handle traffic. Every request returns 200.

```
adityajha@Adityas-MacBook-Pro todo-k8s % kubectl -n todo get rs -w
NAME                  DESIRED   CURRENT   READY   AGE
mongo-646bd6dd45      1         1         1       5m7s
todo-app-74f495ff8b   3         3         3       5m7s
todo-app-d45c5bff     1         0         0       0s
todo-app-d45c5bff     1         0         0       0s
todo-app-d45c5bff     1         1         0       0s
todo-app-d45c5bff     1         1         0       0s
todo-app-d45c5bff     1         1         1       3s
todo-app-d45c5bff     1         1         1       8s
todo-app-74f495ff8b   2         3         3       5m24s
todo-app-74f495ff8b   2         3         3       5m24s
todo-app-d45c5bff     2         1         1       8s
todo-app-74f495ff8b   2         2         2       5m24s
todo-app-d45c5bff     2         1         1       8s
todo-app-d45c5bff     2         2         1       8s
todo-app-d45c5bff     2         2         2       11s
todo-app-74f495ff8b   2         2         2       5m29s
todo-app-d45c5bff     2         2         2       16s
todo-app-74f495ff8b   1         2         2       5m32s
todo-app-74f495ff8b   1         2         2       5m32s
todo-app-d45c5bff     3         2         2       16s
todo-app-74f495ff8b   1         1         1       5m32s
todo-app-d45c5bff     3         2         2       16s
todo-app-d45c5bff     3         3         2       16s
todo-app-d45c5bff     3         3         3       19s
todo-app-74f495ff8b   1         1         1       5m37s
todo-app-d45c5bff     3         3         3       24s
todo-app-74f495ff8b   0         1         1       5m40s
todo-app-74f495ff8b   0         1         1       5m40s
todo-app-74f495ff8b   0         0         0       5m40s
todo-app-74f495ff8b   0         0         0       5m45s
```

Figure 6.2: The new ReplicaSet d45c5bff grows 0 to 3 while the old one 74f495ff8b shrinks 3 to 0, one pod at a time.

```
adityajha@Adityas-MacBook-Pro todo-k8s % kubectl -n todo rollout history deploy/todo-app
kubectl -n todo get rs
deployment.apps/todo-app
REVISION  CHANGE-CAUSE
1         initial deploy
2         Rolling update v1 to v2

NAME                  DESIRED   CURRENT   READY   AGE
mongo-646bd6dd45      1         1         1       7m
todo-app-74f495ff8b   0         0         0       7m
todo-app-d45c5bff     3         3         3       104s
adityajha@Adityas-MacBook-Pro todo-k8s %
```

Figure 6.3: Rollout history with change causes. The old ReplicaSet is kept at 0 replicas so it can be rolled back to.

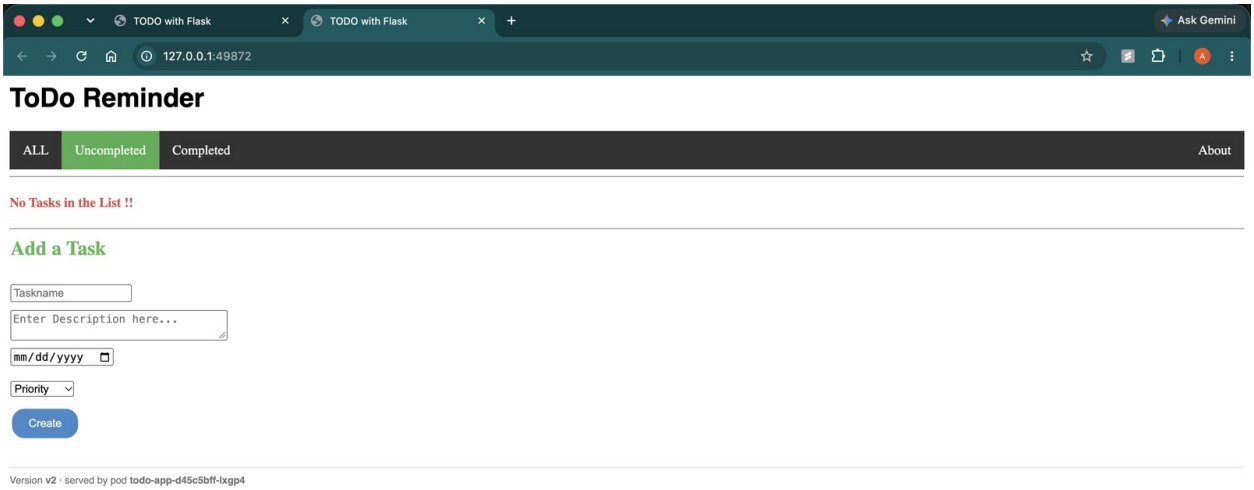


Figure 6.4: The page footer shows version v2, served by a pod from the new ReplicaSet d45c5bff.

Failed rollout 1: application crash (v3-broken)

```
kubectl -n todo set image deploy/todo-app todo-app=thisisadityajha/todo-flask:v3-broken
kubectl -n todo rollout status deploy/todo-app --timeout=180s
kubectl -n todo logs <crashing pod> --previous
kubectl -n todo rollout undo deploy/todo-app
```

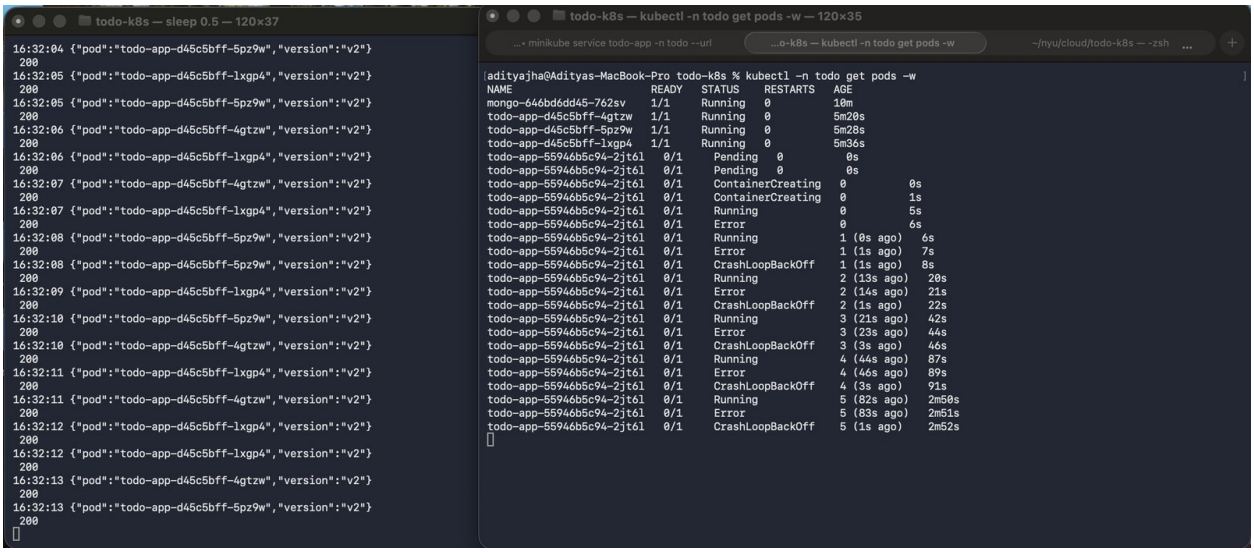


Figure 6.5: Left, users keep getting v2 with 200 on every request. Right, the single v3-broken pod cycles through Running, Error and CrashLoopBackOff with growing back off, while the three v2 pods are never touched.

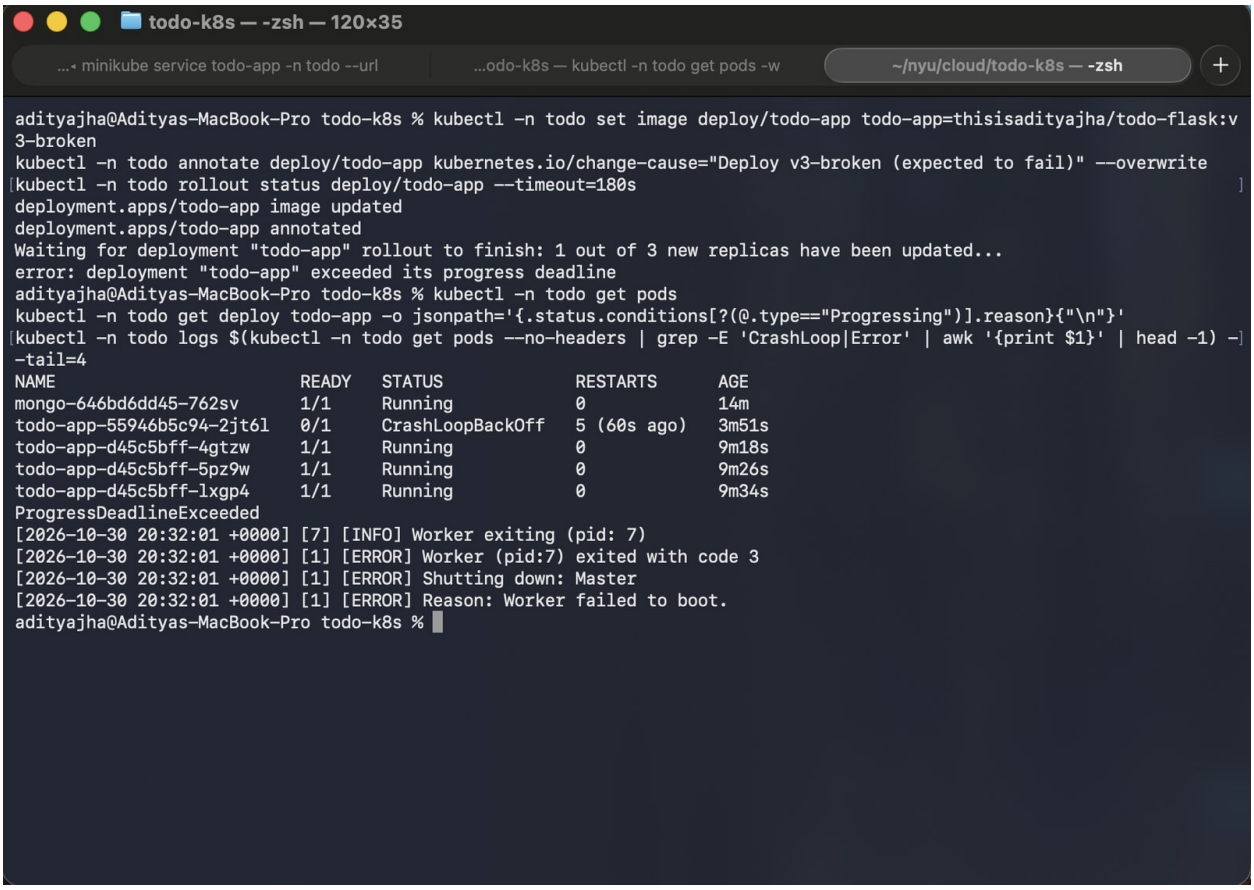


Figure 6.6: After 120 s the rollout is marked ProgressDeadlineExceeded. Logs show gunicorn exiting with Worker failed to boot.

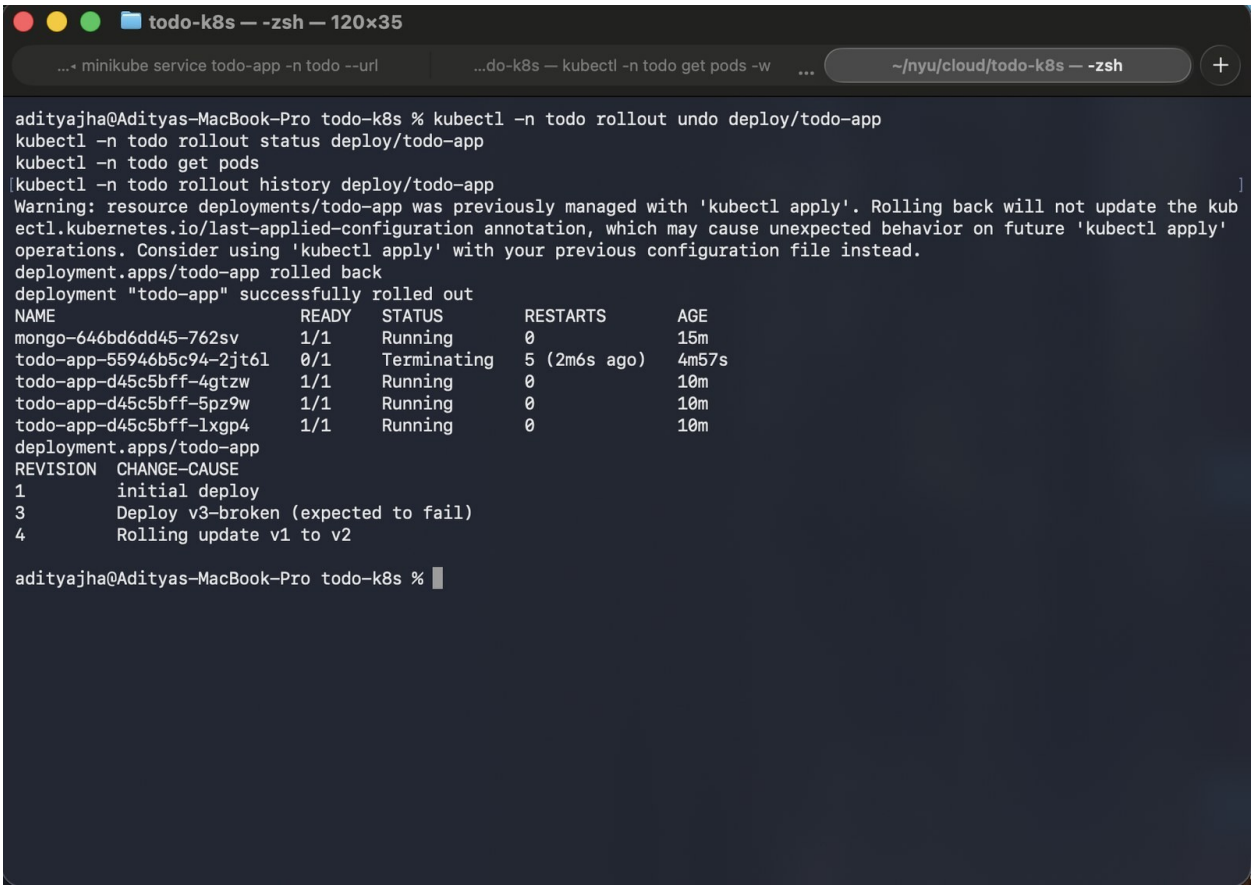


Figure 6.7: kubectl rollout undo deletes the broken pod. Revision 2 reappears as revision 4, because undo applies the old pod template as a new revision.

Failed rollout 2: image tag that does not exist

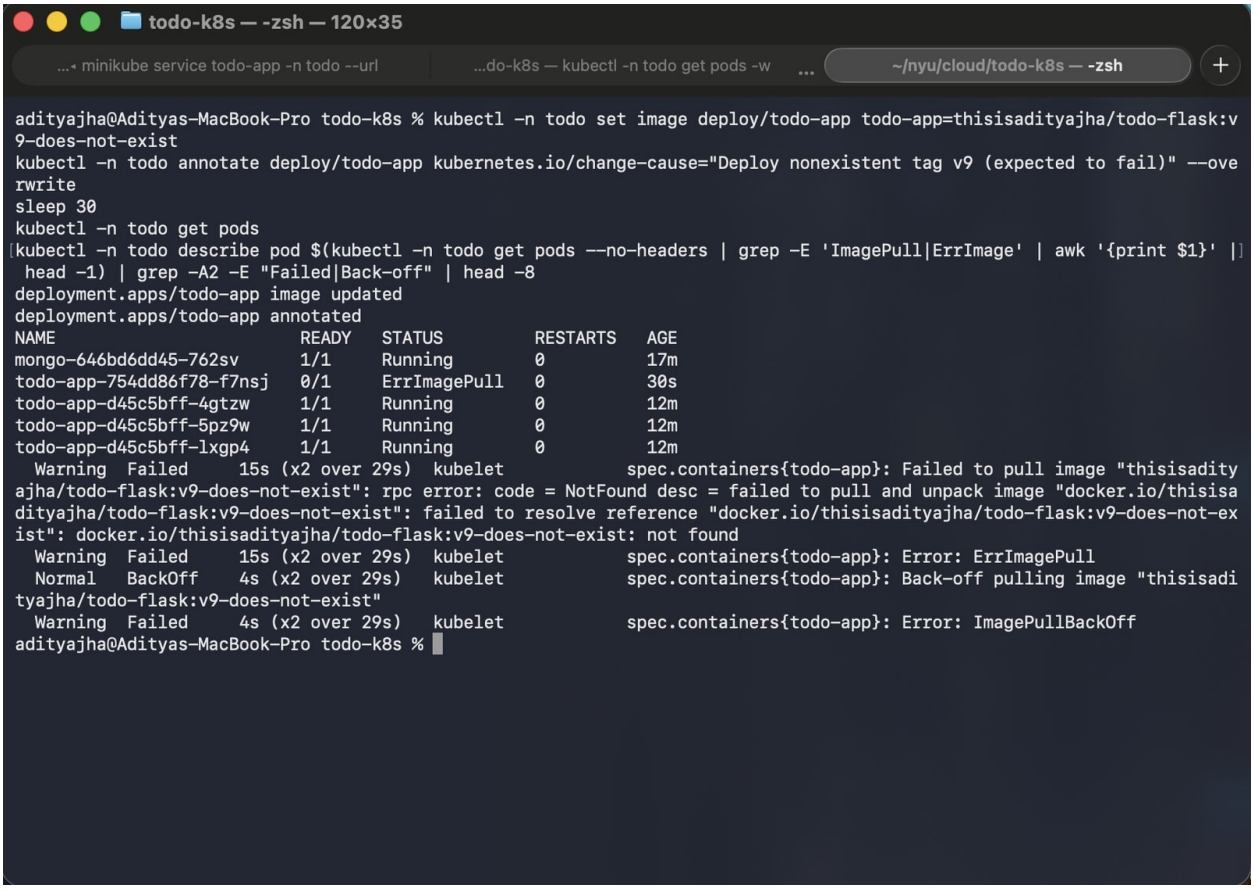


Figure 6.8: Deploying tag v9-does-not-exist. Docker Hub answers not found, the pod goes ErrImagePull then ImagePullBackOff with 0 restarts, and the v2 pods keep serving.

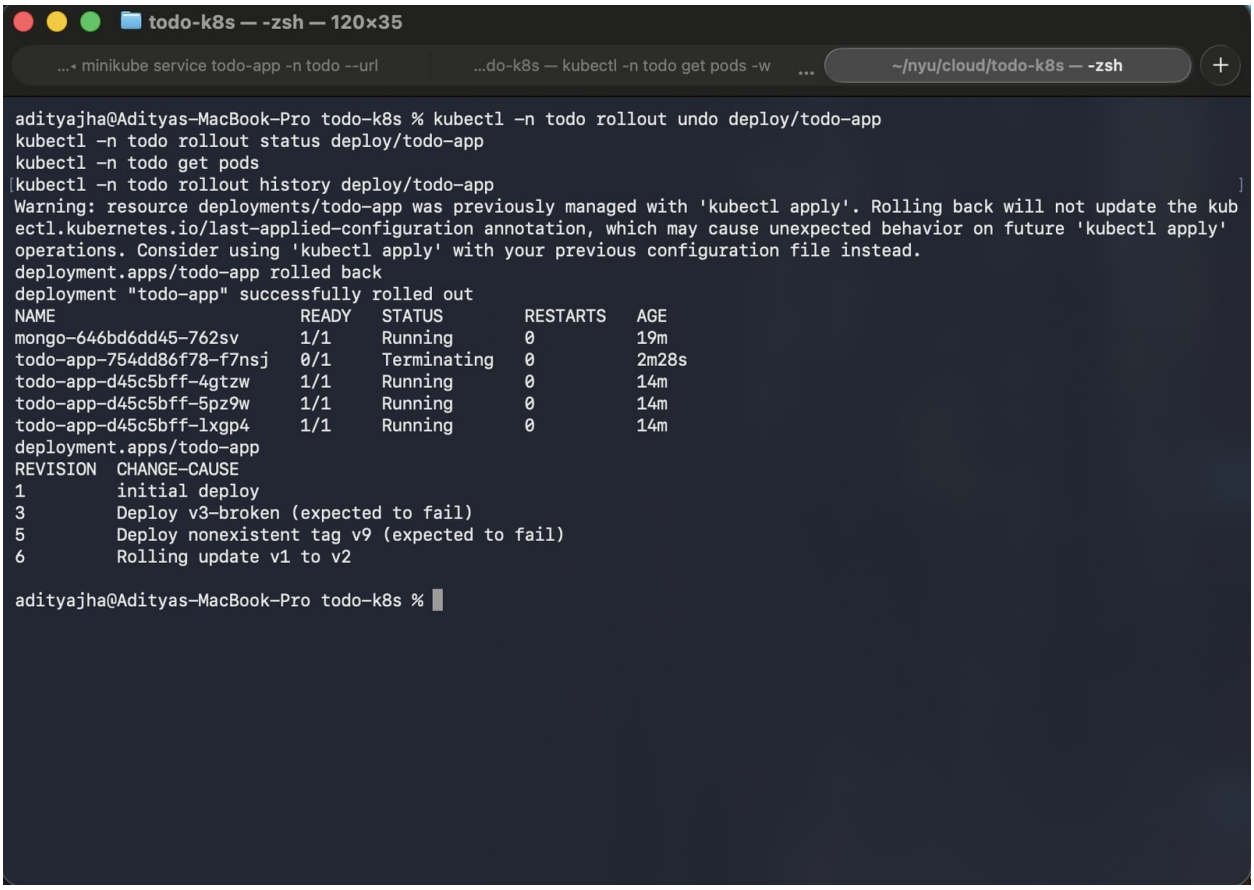


Figure 6.9: A second undo restores v2. The history shows both failed attempts with their change causes.

	v3-broken	**v9-does-not-exist**
Where it fails	Container starts, app crashes on import	Image never downloads
Pod status	CrashLoopBackOff, restarts climb	ImagePullBackOff, 0 restarts
How to diagnose	kubectl logs --previous	kubectl describe pod events
User impact	None, v2 kept serving	None, v2 kept serving

kubectl warns after each undo that the last-applied-configuration annotation is now stale. Because the files already said v2, one kubectl apply -k refreshed the annotation without restarting any pod.

Part 7: Health Monitoring

Each app pod has three probes, and each answers a different question. Liveness asks whether the process is stuck and should be restarted. Readiness asks whether the pod can serve users right now. Startup gives a slow first boot time before liveness starts judging.

Probe	Endpoint	Period	Timeout	Failures allowed	Action on failure
Startup	GET /healthz	3 s	1 s	20 (up to 60 s)	Restart the container. Liveness and readiness wait until it passes.
Liveness	GET /healthz	10 s	2 s	3 (about 30 s)	Restart the container.
Readiness	GET /readyz	5 s	3 s	2 (about 10 s)	Remove the pod from the Service endpoints. No restart.

1. Liveness never touches the database. If it did, a Mongo outage would make every app pod fail liveness and restart over and over, which cannot fix a database problem. Only readiness checks Mongo.
2. A startup probe instead of a fixed initialDelaySeconds. A guessed delay is either too short (restart loops on a slow node) or too long (slow recovery). The startup probe waits only as long as needed.
3. Graceful shutdown. A preStop sleep of 5 s gives the Service time to stop sending traffic before gunicorn begins its graceful exit, so rolling updates drop no requests.

The Mongo pod has its own liveness and readiness probes that run a mongosh ping.

Test	How we caused it	What Kubernetes did
T1 Liveness failure	kubectl exec <pod> -- touch /tmp/unhealthy, so /healthz returns 500	3 liveness failures, then Container todo-app failed liveness probe, will be restarted. RESTARTS went from 0 to 1, and the pod was 1/1 again about 6 s after the restart. The restart cleared the flag file, because the container filesystem is fresh.
T2 Readiness failure	kubectl scale deployment mongo --replicas=0	All 3 app pods went 0/1 within about 10 s with zero restarts. The EndpointSlice showed every address as ready=false. After scaling Mongo back to 1, all pods were 1/1 again within 7 s, with no human action.
T3 Crashing application	A deliberately broken image (import of a module that does not exist) rolled out with kubectl set image	The new pod went Error, then CrashLoopBackOff, with growing back off. kubectl logs --previous showed ModuleNotFoundError. Because maxUnavailable is 0, the 3 old pods kept serving. kubectl rollout undo restored the working version.
T4 Pod deletion	kubectl delete pod (see Part 5)	The replacement passed startup, then readiness, before it joined the Service.

T2 is the key result: with readiness and liveness separated, a database outage takes the app out of the load balancer without a single restart, and recovery is automatic. The Part 7 tests ran on a local build of the same code before v1 was published to Docker Hub, which is why Figure 7.1 names the image todo-flask:dev; Part 6 and Part 8 use the Docker Hub images.

```
(base) pragyaawasthi@MacBookPro todo-k8s % kubectl describe pod -n todo todo-app-6b8bc9b9bb-whb8s | tail -10
                 node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason     Age                  From     Message
  ----     ------     ----                 ----     -------
  Warning  Unhealthy  14m (x33 over 57m)   kubelet  spec.containers{todo-app}: Readiness probe failed: HTTP probe failed with statuscode: 503
  Warning  Unhealthy  41s (x3 over 61s)    kubelet  spec.containers{todo-app}: Liveness probe failed: HTTP probe failed with statuscode: 500
  Normal   Killing    41s                  kubelet  spec.containers{todo-app}: Container todo-app failed liveness probe, will be restarted
  Normal   Pulled     35s (x2 over 62m)    kubelet  spec.containers{todo-app}: Container image "todo-flask:dev" already present on machine and can be accessed by the pod
  Normal   Created    35s (x2 over 62m)    kubelet  spec.containers{todo-app}: Container created
  Normal   Started    35s (x2 over 62m)    kubelet  spec.containers{todo-app}: Container started
```

Figure 7.1: Events for pod whb8s. Liveness failed 3 times with status 500 (x3 over 61 s), then the kubelet logged Container todo-app failed liveness probe, will be restarted, and started a fresh container. The older readiness failures with 503 are from the Mongo outage tests, when this pod left the Service but was never restarted.

```
(base) pragyaawasthi@MacBookPro todo-k8s % kubectl exec -n todo todo-app-6b8bc9b9bb-whb8s -- touch /tmp/unhealthy
[kubectl get pods -n todo -l app=todo-app -w
NAME                        READY   STATUS    RESTARTS      AGE
todo-app-6b8bc9b9bb-dxt8g   1/1     Running   0             72s
todo-app-6b8bc9b9bb-fqfkv   1/1     Running   1 (49m ago)   61m
todo-app-6b8bc9b9bb-whb8s   1/1     Running   0             61m
todo-app-6b8bc9b9bb-whb8s   0/1     Running   1 (0s ago)    62m
todo-app-6b8bc9b9bb-whb8s   0/1     Running   1 (3s ago)    62m
todo-app-6b8bc9b9bb-whb8s   0/1     Running   1 (6s ago)    62m
todo-app-6b8bc9b9bb-whb8s   1/1     Running   1 (6s ago)    62m
```

Figure 7.2: After the flag file was created, whb8s restarted (RESTARTS 0 to 1), was 0/1 for about 6 s while readiness checked it, then returned to 1/1. The restart cleared the flag, so it did not loop.

```
(base) pragyaawasthi@MacBookPro todo-k8s % kubectl scale deployment mongo -n todo --replicas=0
sleep 20
kubectl get pods -n todo
kubectl get endpointslices -n todo -l kubernetes.io/service-name=todo-app \
  -o jsonpath='{range .items[*].endpoints[*]}{.addresses[0]}{"  ready="}{.conditions.ready}{"\n"}{end}'
[deployment.apps/mongo scaled
NAME                        READY   STATUS    RESTARTS      AGE
todo-app-6b8bc9b9bb-4r9jz   0/1     Running   0             13m
todo-app-6b8bc9b9bb-fqfkv   0/1     Running   1 (37m ago)   49m
todo-app-6b8bc9b9bb-whb8s   0/1     Running   0             49m
10.244.0.12  ready=false
10.244.0.11  ready=false
10.244.0.28  ready=false
```

Figure 7.3: 20 s after scaling Mongo to 0, all 3 app pods are 0/1 but still Running, and the EndpointSlice marks every address ready=false. No new restarts happened.

```
(base) pragyaawasthi@MacBookPro todo-k8s % kubectl set image deployment/todo-app -n todo todo-app=todo-flask:broken
[kubectl get pods -n todo -l app=todo-app -w
deployment.apps/todo-app image updated
NAME                        READY   STATUS              RESTARTS      AGE
todo-app-5dc4d85694-wcrxq   0/1     ContainerCreating   0             0s
todo-app-6b8bc9b9bb-dxt8g   1/1     Running             0             3m1s
todo-app-6b8bc9b9bb-fqfkv   1/1     Running             1 (51m ago)   63m
todo-app-6b8bc9b9bb-whb8s   1/1     Running             1 (81s ago)   63m
todo-app-5dc4d85694-wcrxq   0/1     ContainerCreating   0             1s
todo-app-5dc4d85694-wcrxq   0/1     Running             0             1s
todo-app-5dc4d85694-wcrxq   0/1     Error               0             2s
todo-app-5dc4d85694-wcrxq   0/1     Running             1 (0s ago)    2s
todo-app-5dc4d85694-wcrxq   0/1     Error               1 (1s ago)    3s
todo-app-5dc4d85694-wcrxq   0/1     CrashLoopBackOff    1 (2s ago)    4s
todo-app-5dc4d85694-wcrxq   0/1     Running             2 (12s ago)   14s
todo-app-5dc4d85694-wcrxq   0/1     Error               2 (14s ago)   16s
todo-app-5dc4d85694-wcrxq   0/1     CrashLoopBackOff    2 (4s ago)    19s
todo-app-5dc4d85694-wcrxq   0/1     Running             3 (24s ago)   39s
todo-app-5dc4d85694-wcrxq   0/1     Error               3 (26s ago)   41s
todo-app-5dc4d85694-wcrxq   0/1     CrashLoopBackOff    3 (2s ago)    42s
[^C%
(base) pragyaawasthi@MacBookPro todo-k8s % kubectl logs -n todo todo-app-5dc4d85694-wcrxq --previous | tail -5
ModuleNotFoundError: No module named 'this_module_does_not_exist'
[2026-10-29 18:21:43 +0000] [7] [INFO] Worker exiting (pid: 7)
[2026-10-29 18:21:43 +0000] [1] [ERROR] Worker (pid:7) exited with code 3
[2026-10-29 18:21:43 +0000] [1] [ERROR] Shutting down: Master
[2026-10-29 18:21:43 +0000] [1] [ERROR] Reason: Worker failed to boot.
```

Figure 7.4: After kubectl set image to the broken image, the new pod wcrxq loops through Running, Error and CrashLoopBackOff, with the wait growing each time (2 s, 12 s, 24 s). The 3 old pods stay 1/1. kubectl logs --previous shows ModuleNotFoundError.



Figure 7.5: kubectl rollout undo switches the Deployment back to the working ReplicaSet. The crashing pod is Terminating and the 3 healthy pods were never interrupted.

Part 8: Alerting with Prometheus and Slack (Extra Credit)

Prometheus detects every failure from Part 7 through 8 alert rules labelled team: todo, and Alertmanager sends only those to a private Slack channel, with a FIRING message when a problem starts and a RESOLVED message when it ends. Both halves were tested end to end on Minikube.

8.1 Detection: Prometheus

We installed the kube-prometheus-stack Helm chart as release mon in a monitoring namespace. It brings the Prometheus Operator, Prometheus, Alertmanager, kube-state-metrics and node-exporter. monitoring/values.yaml sets a 15 s scrape and evaluation interval and 2 days of retention, picks up every ServiceMonitor and PrometheusRule, and turns off Grafana and the control plane targets Minikube cannot expose, so they do not raise permanent false alerts.

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
kubectl create namespace monitoring
kubectl create secret generic alertmanager-slack -n monitoring --from-literal=url="$SLACK_URL"
helm upgrade --install mon prometheus-community/kube-prometheus-stack -n monitoring \
  -f monitoring/values.yaml -f monitoring/alertmanager-slack.yaml
kubectl apply -f monitoring/todo-servicemonitor.yaml -f monitoring/todo-alert-rules.yaml
```

monitoring/todo-servicemonitor.yaml scrapes /metrics every 15 s on every pod behind the todo-app Service; the Target health page showed 3/3 up.

Alert	Fires when	Severity	Catches
TodoLivenessProbeFailures	More than 2 liveness failures on a pod in 2 min	critical	T1, E2
TodoReadinessProbeFailures	More than 3 readiness failures on a pod in 2 min, held for 1 min	warning	T2, E3
TodoContainerRestarted	Any container restart in 5 min	warning	T1, T3, E2
TodoCrashLooping	A container in CrashLoopBackOff for 1 min	critical	T3
TodoReplicasMismatch	Available replicas differ from desired, per Deployment	warning	Pod deletion, outages
TodoNewPodStarted	A pod in todo started less than 2 min ago	info	Pod deletion, scaling, rollout
TodoMongoDown	Mongo has 0 available replicas for 30 s; kept firing 3 min after recovery	critical	T2, E3
TodoHigh5xxRate	Over 20% of app requests return 5xx for 1 min	warning	Errors seen by users

The probe alerts read the kubelet's prober_probe_total counter, and the pod and Deployment alerts read kube-state-metrics. They use increase(...[2m]) and pod start time instead of instant values, because a failure shorter than one 15 s scrape could otherwise be missed. Because increase() extrapolates across scrapes, alert values are often fractional (for example 2.29 failures).

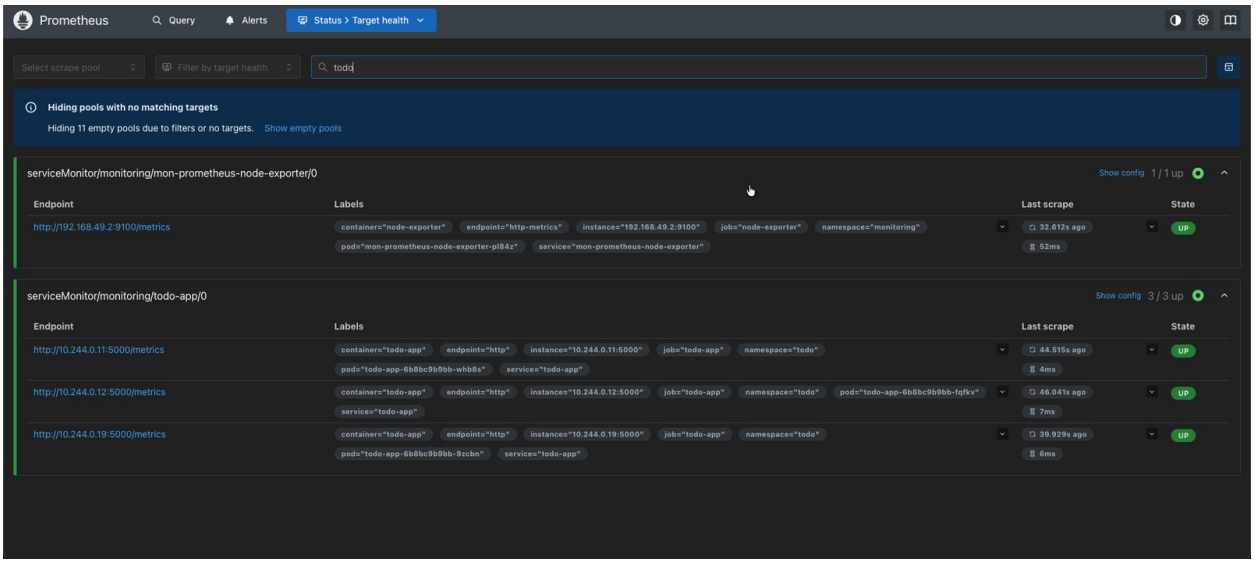


Figure 8.1: Target health filtered to todo. The ServiceMonitor finds all 3 app pods, each UP.

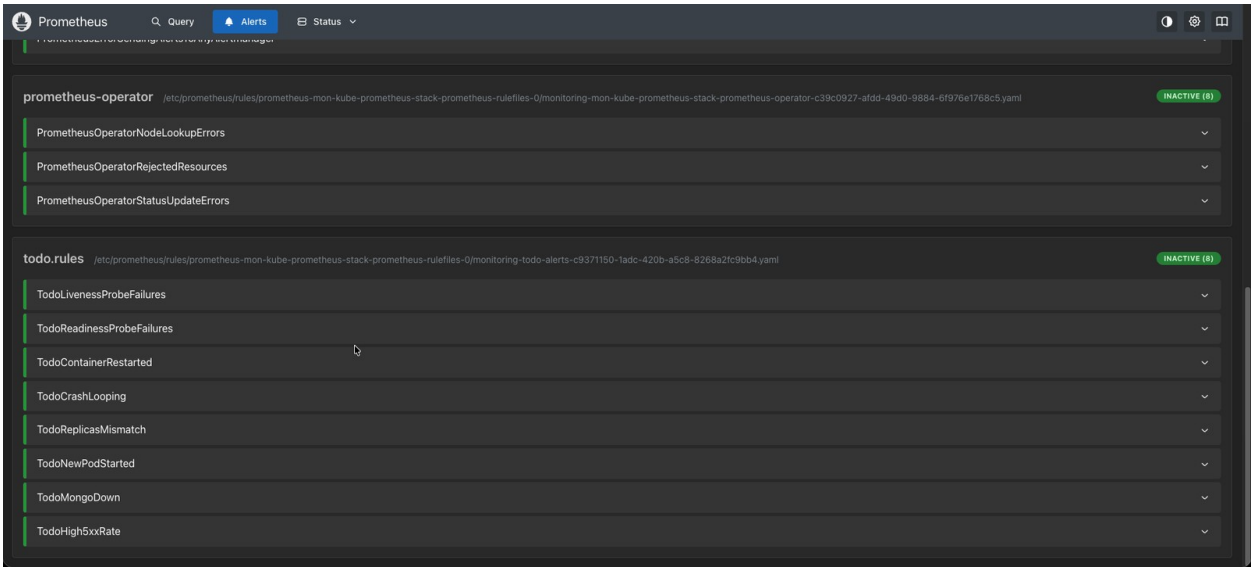


Figure 8.2: The todo.rules group loaded, INACTIVE (8), while nothing is broken.

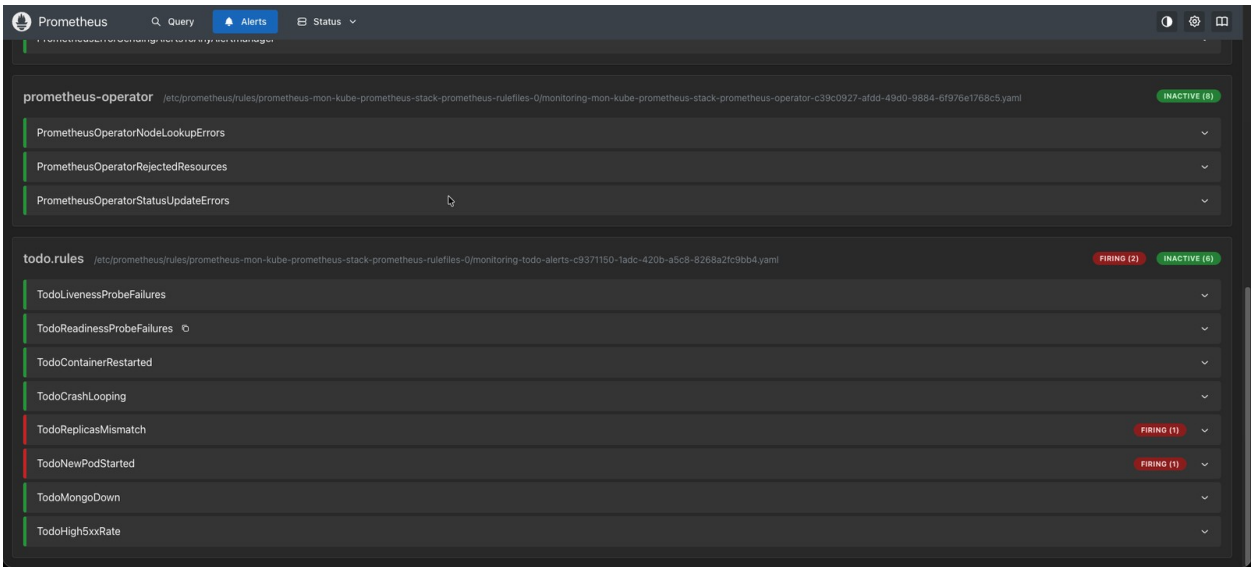


Figure 8.3: After deleting a pod, TodoReplicasMismatch and TodoNewPodStarted are both Firing.

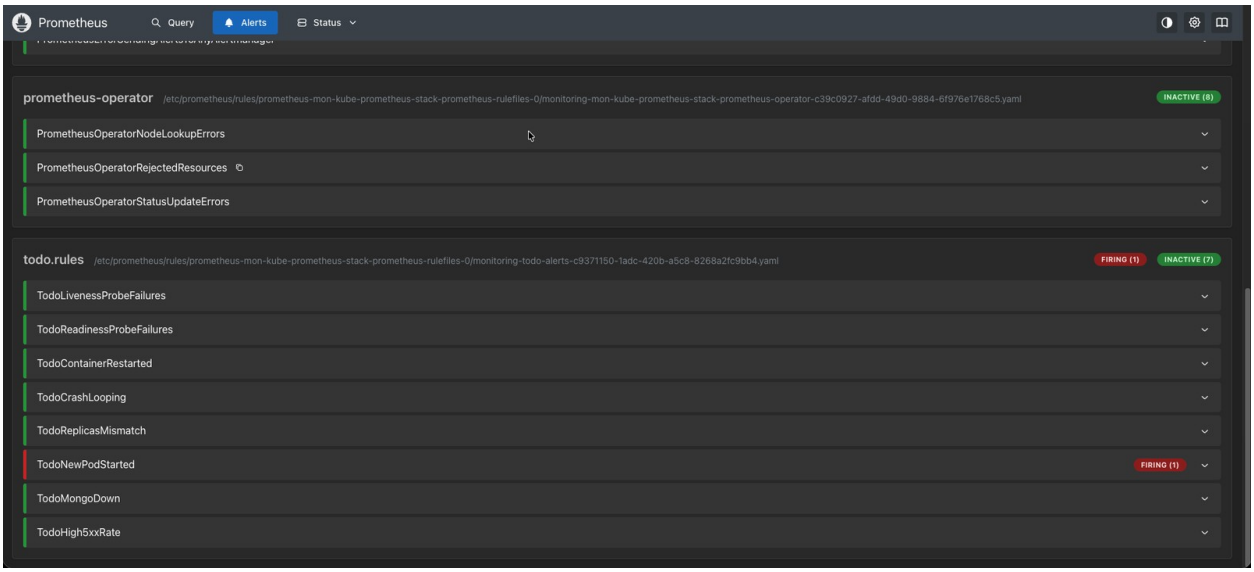


Figure 8.4: Seconds later the replacement is ready, so TodoReplicasMismatch has resolved by itself. TodoNewPodStarted keeps firing until the pod is 2 minutes old.

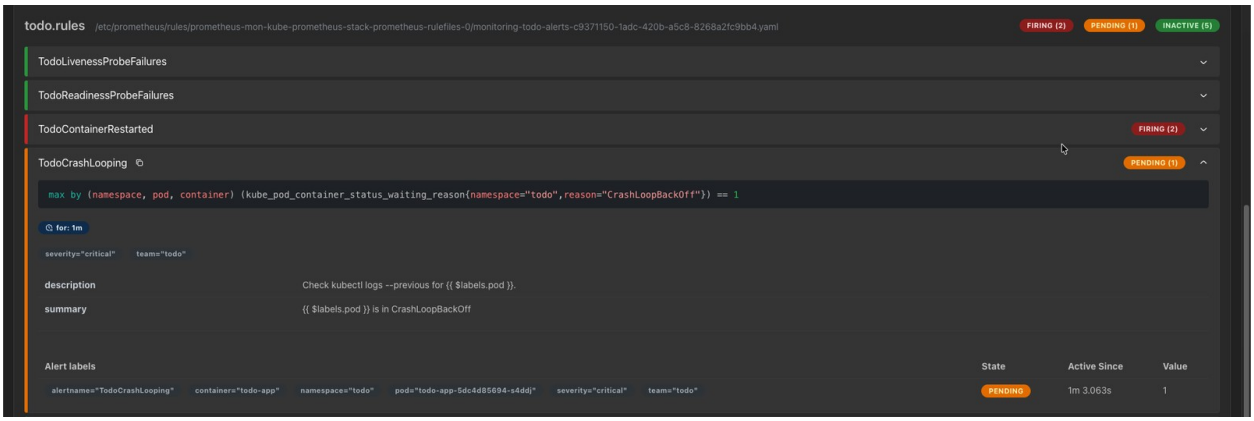


Figure 8.5: TodoCrashLooping in the Pending state, expanded to show its PromQL, the for: 1m wait, the severity="critical" and team="todo" labels, and the crashing pod it matched.



Figure 8.6: A minute later, TodoCrashLooping is Firing together with TodoContainerRestarted and TodoNewPodStarted: three independent signals for one bad rollout.

8.2 Notification: Alertmanager and Slack

A Slack app k8s-alertmanager posts through an incoming webhook to the private channel #k8s-alerts in our own workspace cloud-a2-todo, never a class channel. The webhook URL is stored only in the Kubernetes Secret alertmanager-slack, mounted into Alertmanager as a file, so it never appears in Git, Helm values or this report.

```
alertmanager:
  alertmanagerSpec:
    secrets: [alertmanager-slack]
  config:
    route:
      receiver: "null"
      group_by: [alertname, namespace, pod]
      group_wait: 10s
      group_interval: 1m
      repeat_interval: 1h
      routes:
        - receiver: "null"
          matchers: ['alertname = "Watchdog"']
        - receiver: "null"
          matchers: ['alertname = "InfoInhibitor"']
        - receiver: slack-todo
          matchers: ['team = "todo"']
    inhibit_rules:
      - source_matchers: ['alertname = "TodoMongoDown"']
```

```
    target_matchers: ['alertname = "TodoReadinessProbeFailures"']
    equal: [namespace]
receivers:
  - name: "null"
  - name: slack-todo
    slack_configs:
      - api_url_file: /etc/alertmanager/secrets/alertmanager-slack/url
        channel: "#k8s-alerts"
        send_resolved: true
```

- Routing: the chart's built in alerts (Watchdog, KubePodNotReady and others) go to the null receiver; only alerts labelled team=todo reach Slack.
- send_resolved: true, so every incident ends with a RESOLVED message.
- Inhibition: while TodoMongoDown fires, the readiness alerts it causes are suppressed, so Slack gets one root cause alert instead of four.
- The Slack message shows the pod, or the Deployment when an alert is about a whole Deployment.

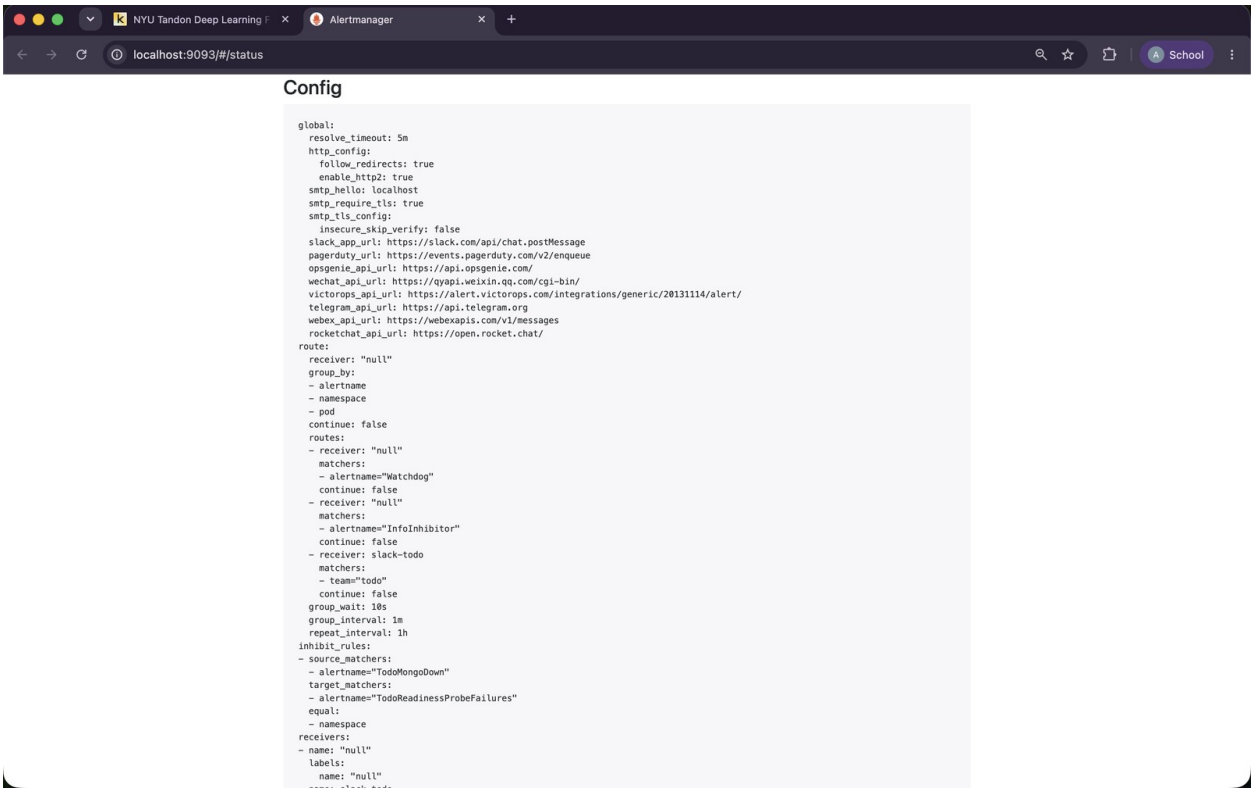


Figure 8.7: Alertmanager status page: the loaded routes, with Watchdog and InfoInhibitor sent to null and team="todo" sent to slack-todo.

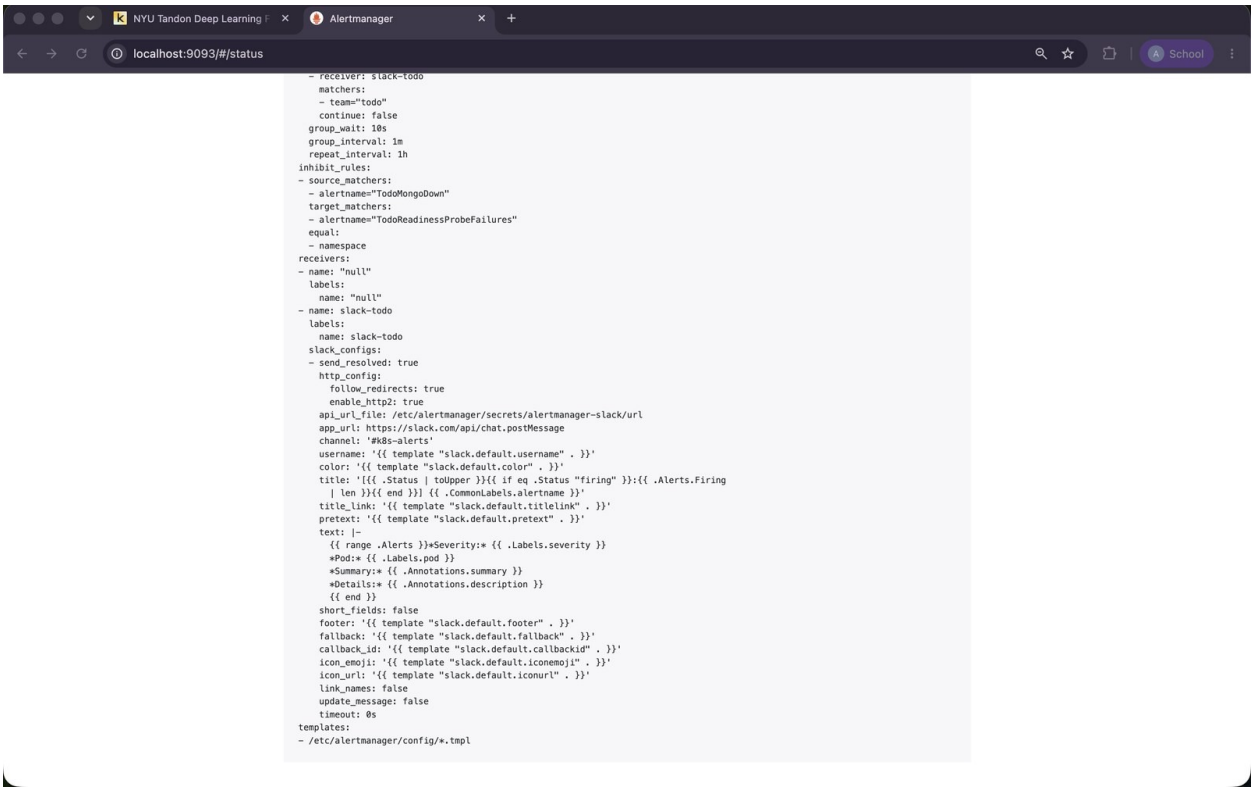


Figure 8.8: The inhibit rule and the slack-todo receiver. The webhook is read from api_url_file, a mounted Secret, so the URL is not in the configuration.

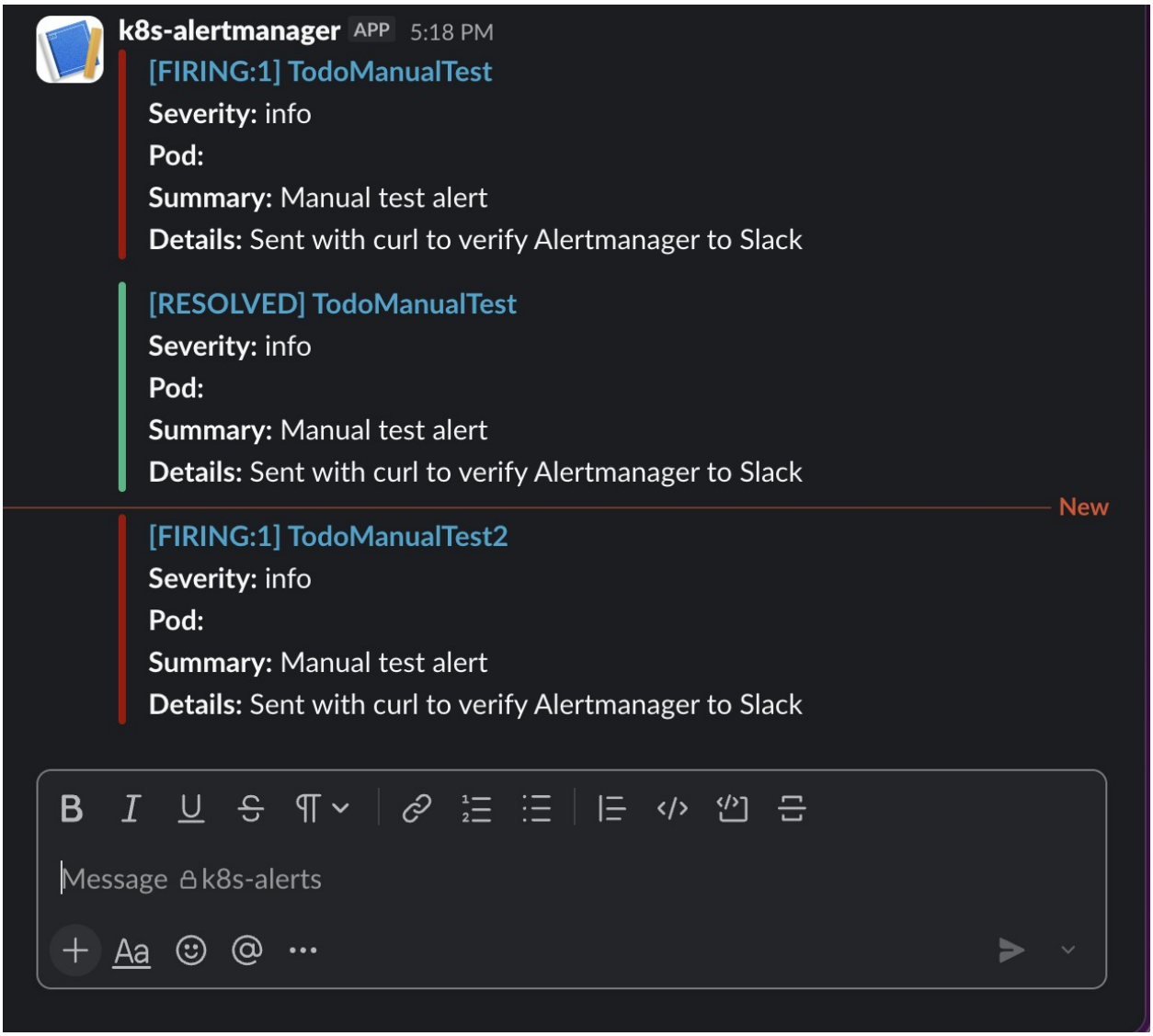


Figure 8.9: A synthetic alert posted with curl to the Alertmanager API arrives in Slack as FIRING, then RESOLVED after resolve_timeout. This tested the Alertmanager to Slack path on its own.

End to end tests

We ran three end to end tests (E1 to E3). Each one causes a real failure on the cluster and checks the whole chain: Prometheus fires the rule, Alertmanager routes it, and Slack shows FIRING and then RESOLVED.

Test	Failure we caused	Alerts expected in Slack	What it proves
E1	Deleted an app pod (kubectl delete pod)	TodoNewPodStarted, TodoReplicasMismatch	The handout's required test: a pod deletion reaches Slack
E2	Made one pod fail its liveness probe (POST /chaos/fail-liveness)	TodoLivenessProbeFailures, TodoContainerRestarted	An alert on failing probes exceeding a threshold, as the handout asks
E3	Scaled MongoDB to 0 replicas for 2 minutes	TodoMongoDown, TodoReplicasMismatch, and no readiness alerts	Routing and inhibition: one root cause alert instead of four

E1: Pod deletion (the handout's test)

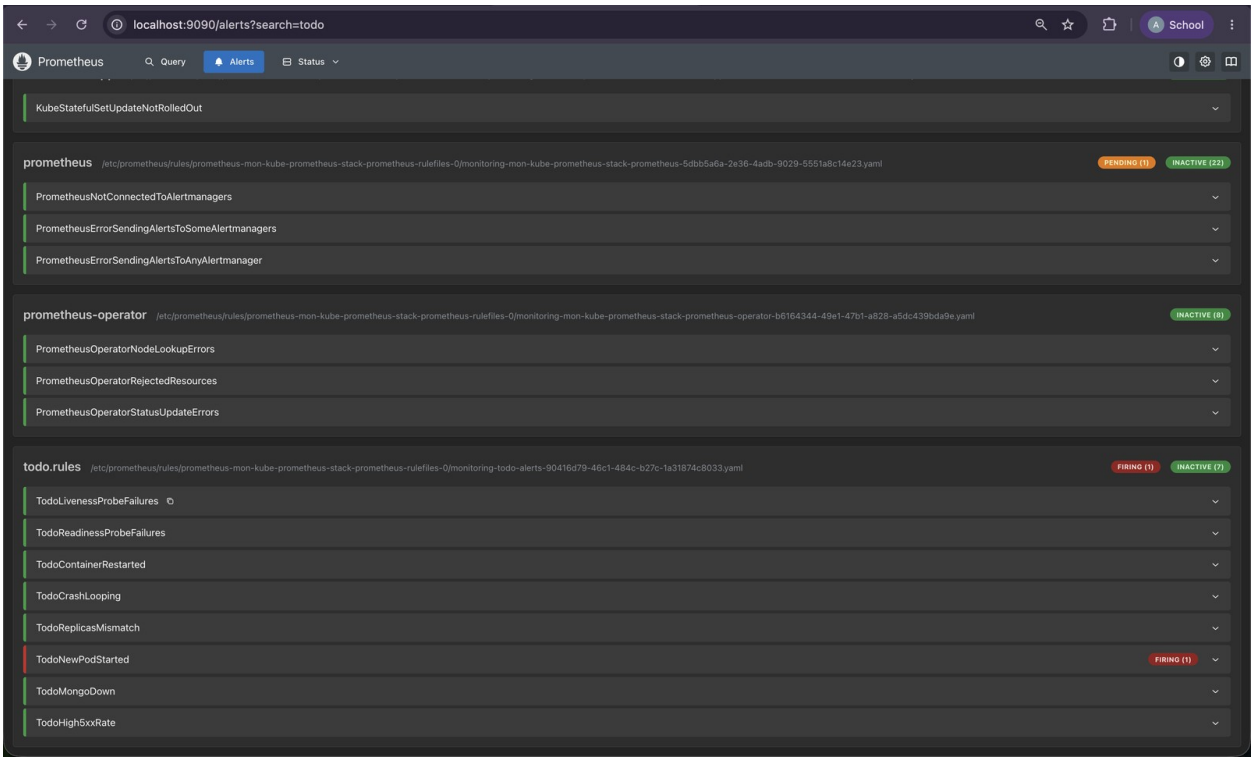


Figure 8.10: After kubectl delete pod, TodoNewPodStarted is Firing in Prometheus; TodoReplicasMismatch already resolved because the replacement was ready within seconds.

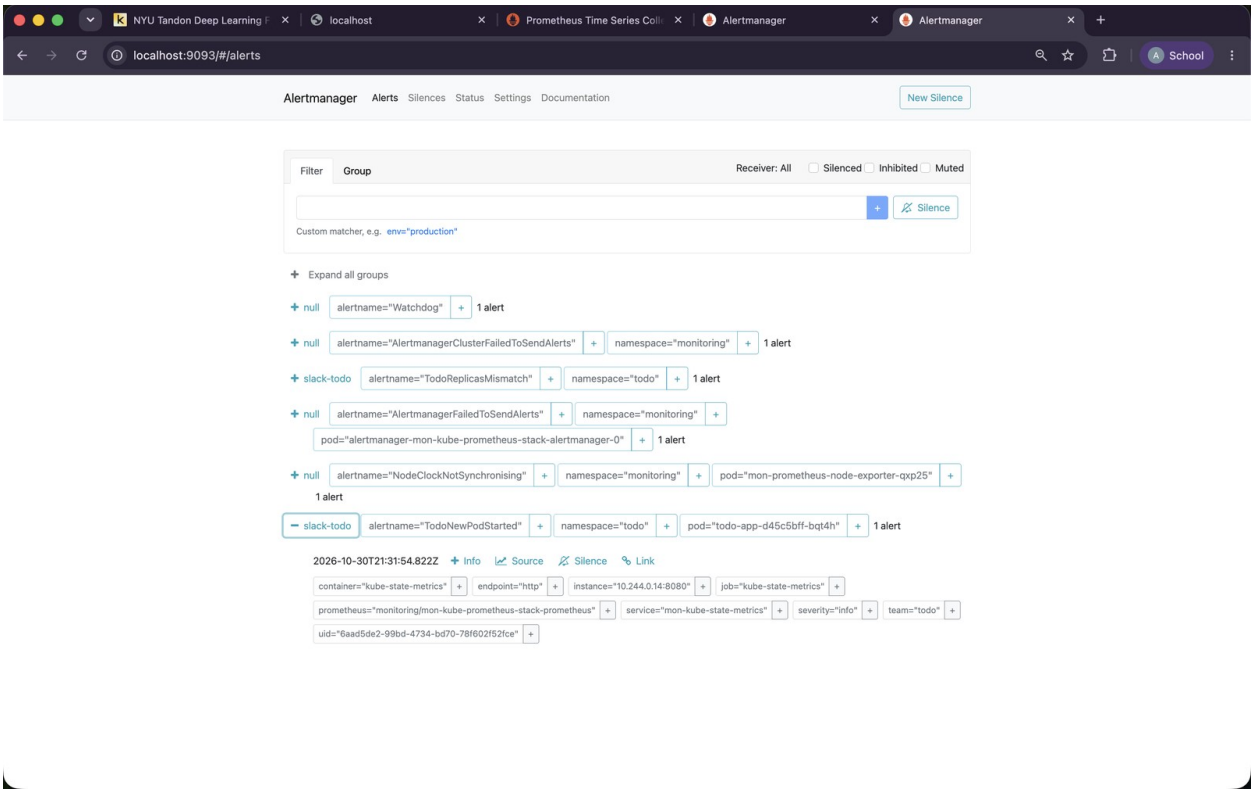


Figure 8.11: Alertmanager groups chart alerts under null and team=todo alerts under slack-todo. Expanded: TodoNewPodStarted for the replacement pod bqt4h, with the team="todo" label that routed it.

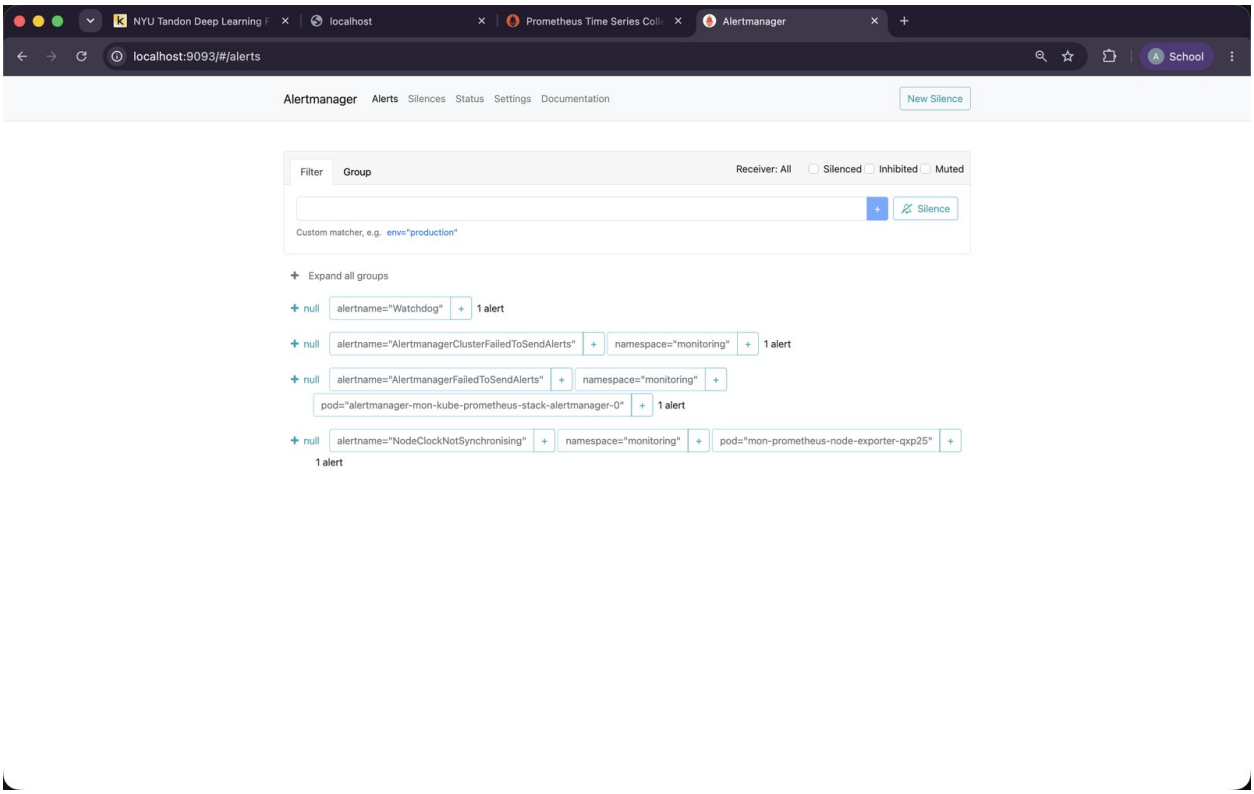


Figure 8.12: Built in chart alerts such as Watchdog are grouped under the null receiver, so they never reach Slack.

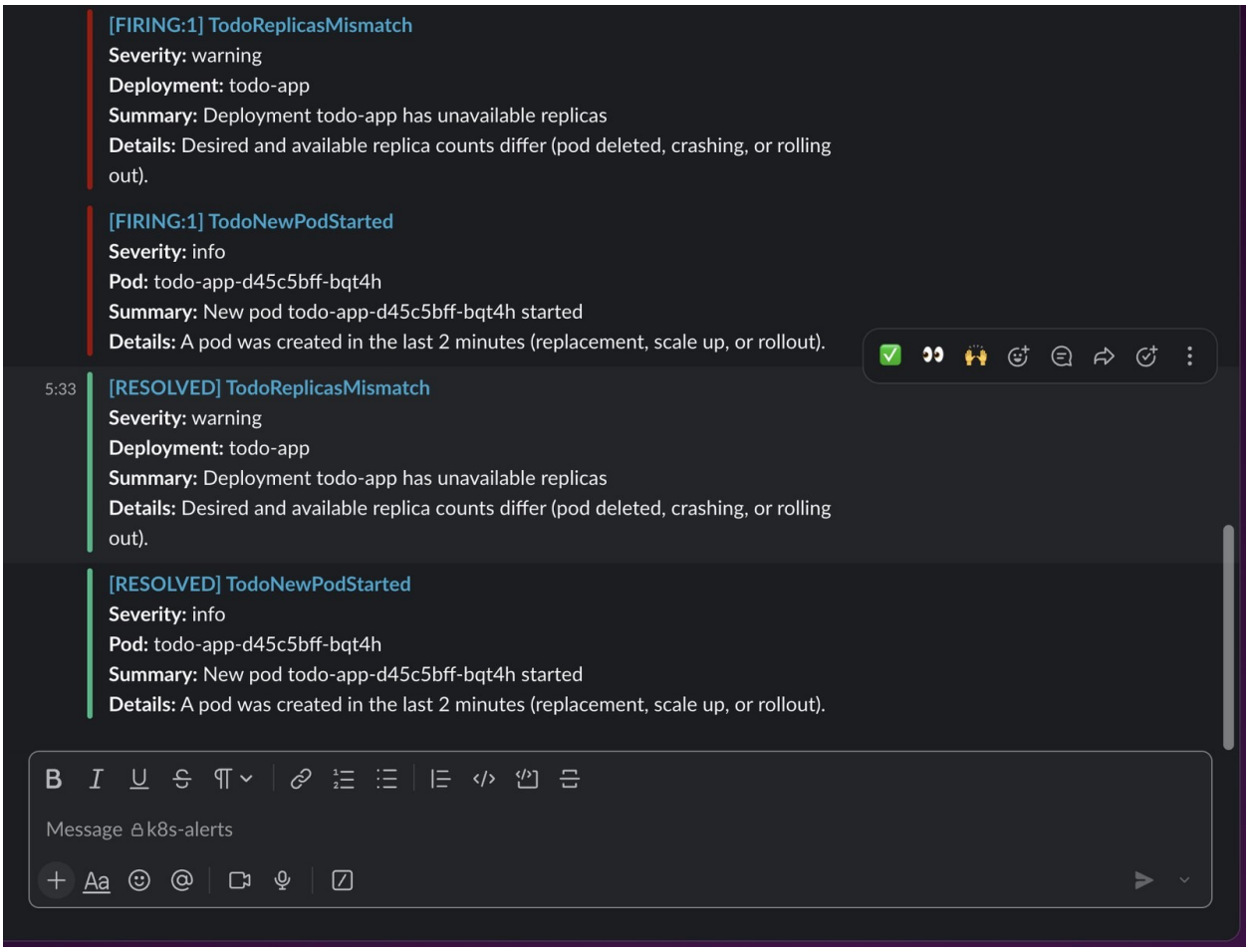


Figure 8.13: E1 in Slack. The deletion is detected by two independent signals, a Deployment below its desired replicas and a new pod started. Both arrive as FIRING and resolve automatically.

E2: Liveness failure (probe threshold)

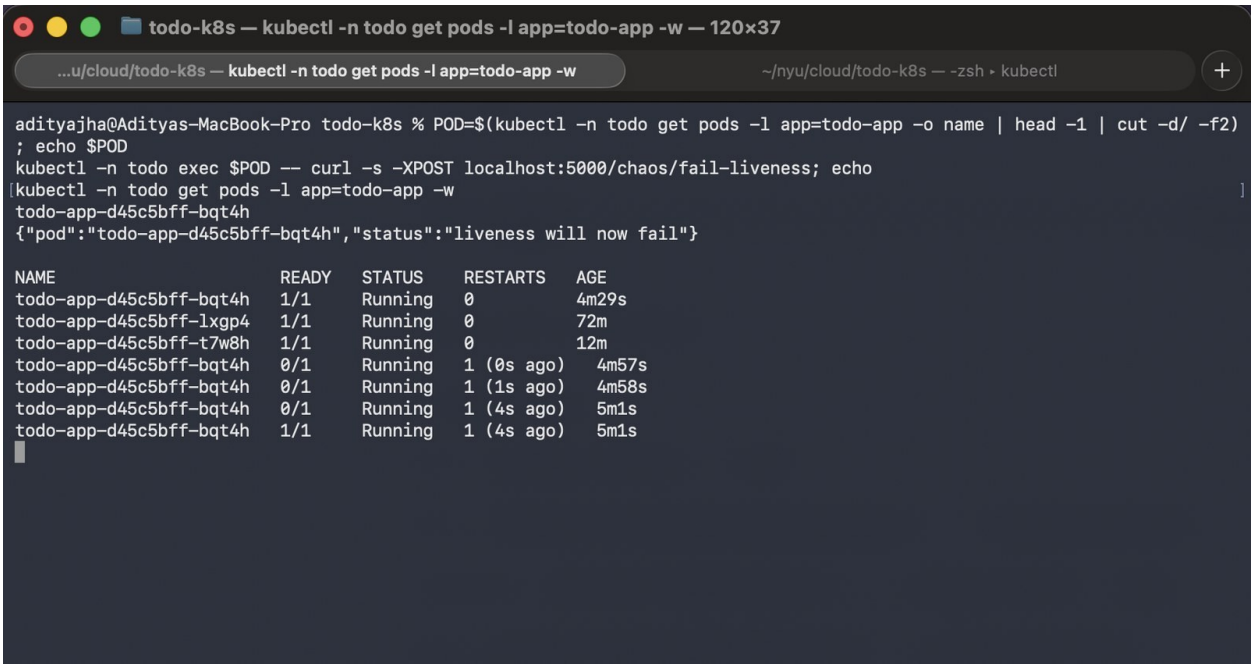


Figure 8.14: POST /chaos/fail-liveness on pod bqt4h. About 28 s later the kubelet restarts it (RESTARTS 0 to 1, same pod name) and it is 1/1 again 4 s later.

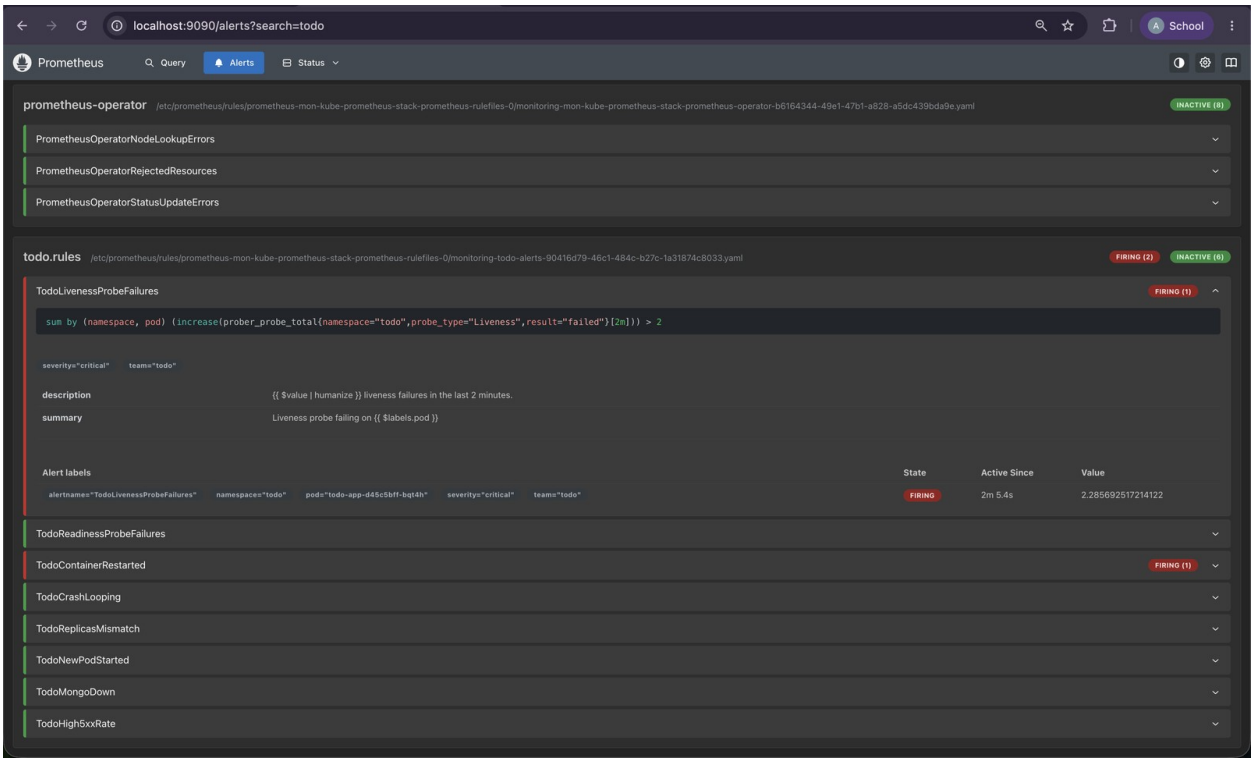


Figure 8.15: TodoLivenessProbeFailures fires when failed liveness probes on bqt4h exceed 2 in 2 minutes (value 2.29), with TodoContainerRestarted firing for the restart that followed.

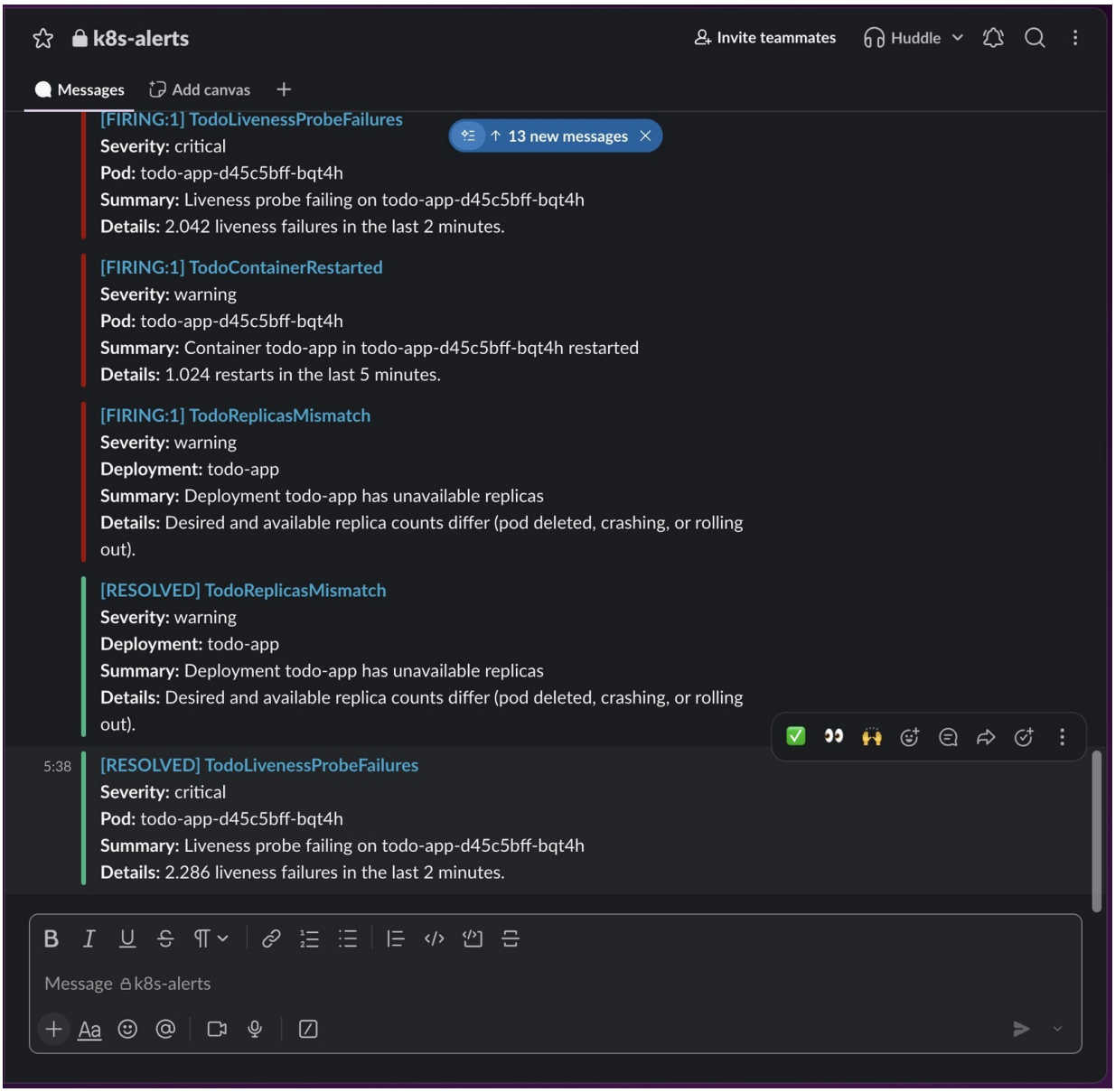


Figure 8.16: E2 in Slack: critical liveness alert and the restart alert for pod bqt4h. The short not ready window after the restart also trips TodoReplicasMismatch.

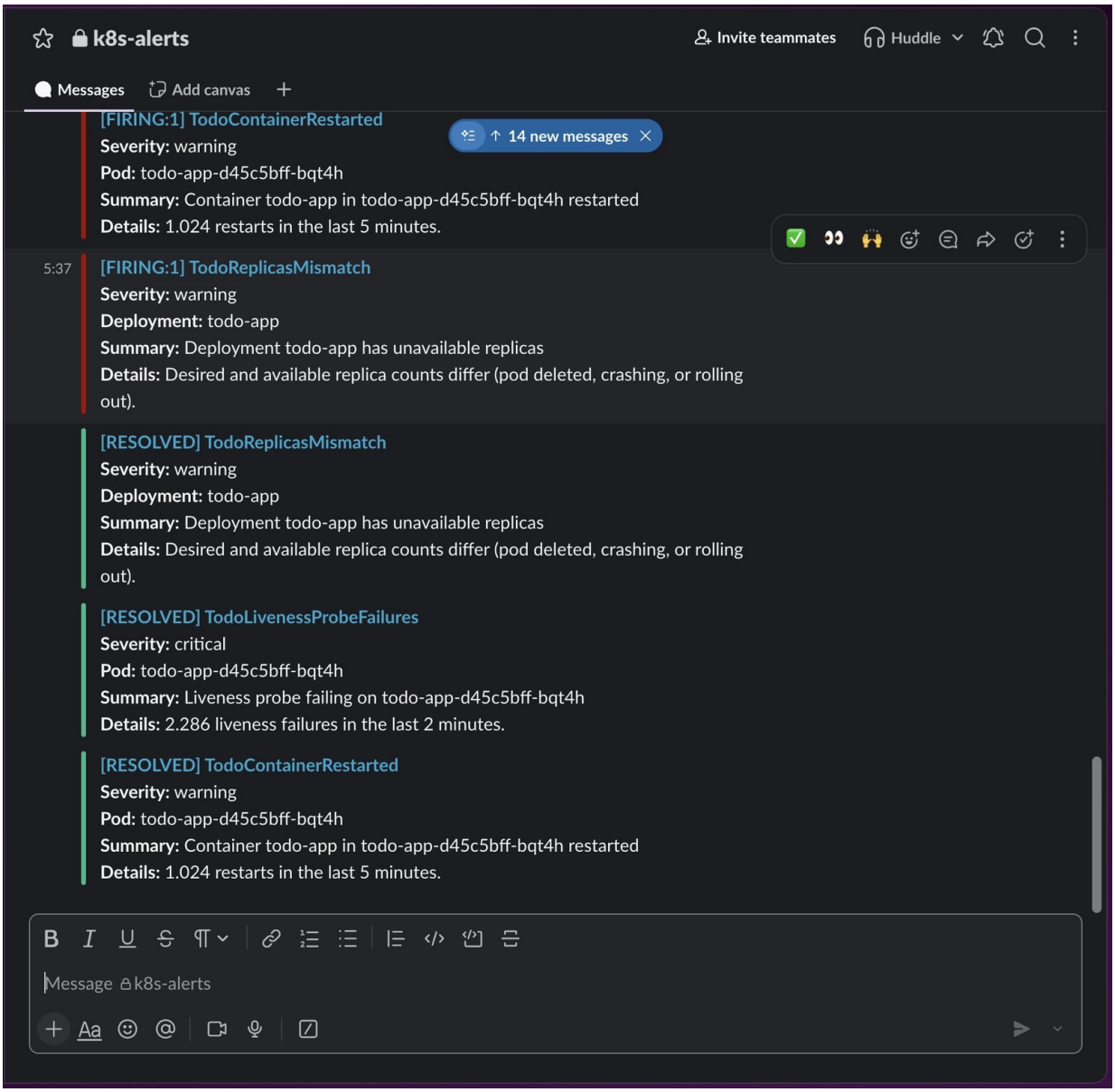


Figure 8.17: All three alerts resolve on their own once the container is healthy.

E3: MongoDB down (routing and inhibition)

Scaling Mongo to 0 makes Prometheus fire the root cause (TodoMongoDown) and its symptoms (readiness failures on all 3 app pods). Our first runs showed that inhibition only suppresses a symptom while its cause is active: readiness alerts reached Slack once before TodoMongoDown fired, and once after it resolved. We fixed both ends: TodoReadinessProbeFailures now waits for: 1m (longer than TodoMongoDown's 30 s), and TodoMongoDown has keep_firing_for: 3m (longer than the readiness rule's 2 minute window). The final run below posted no readiness message at all.

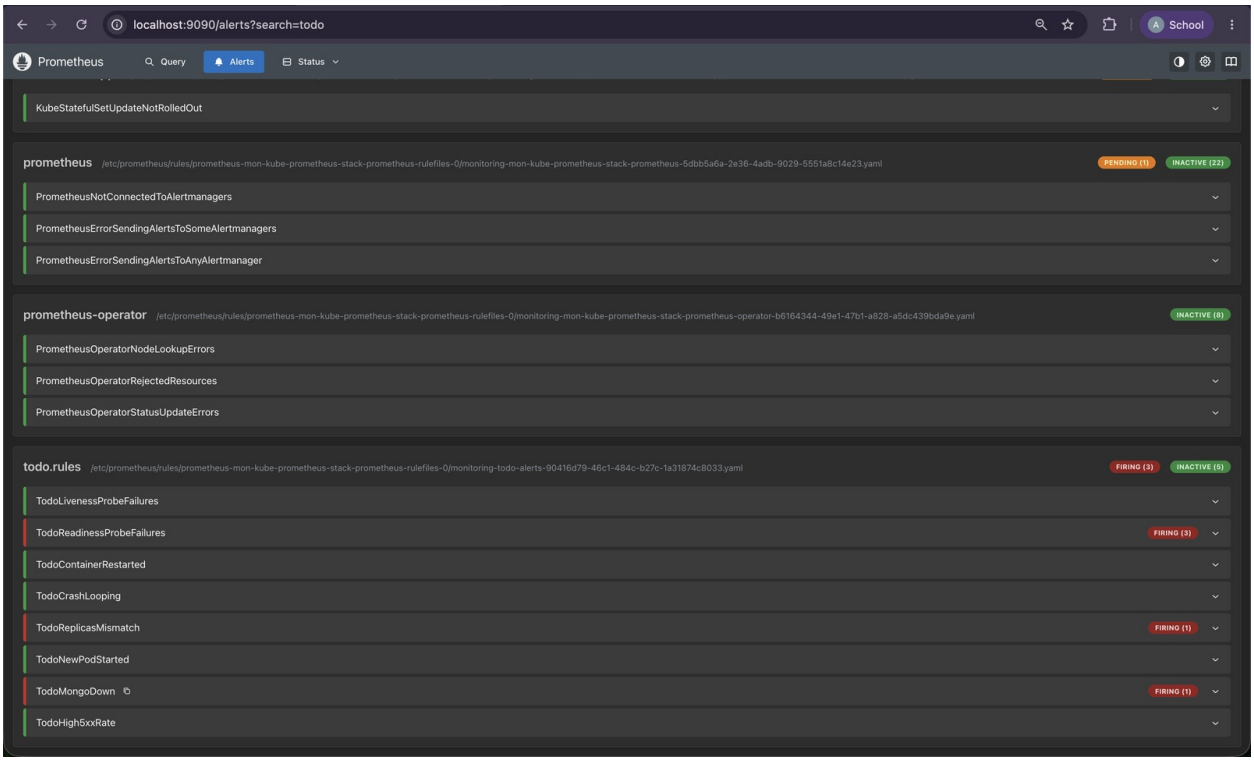


Figure 8.18: With Mongo at 0, Prometheus fires TodoMongoDown, TodoReplicasMismatch and TodoReadinessProbeFailures for all 3 pods.

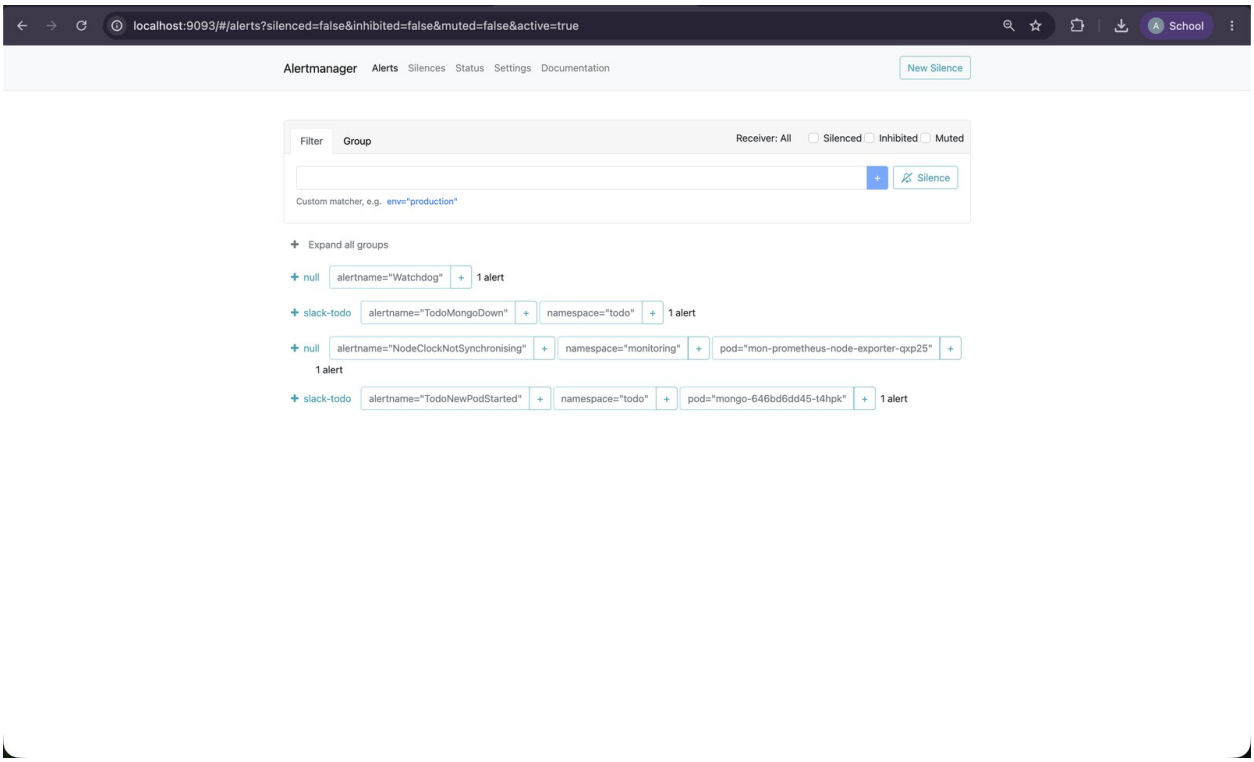


Figure 8.19: Alertmanager with Inhibited unticked: slack-todo shows only TodoMongoDown and TodoNewPodStarted.

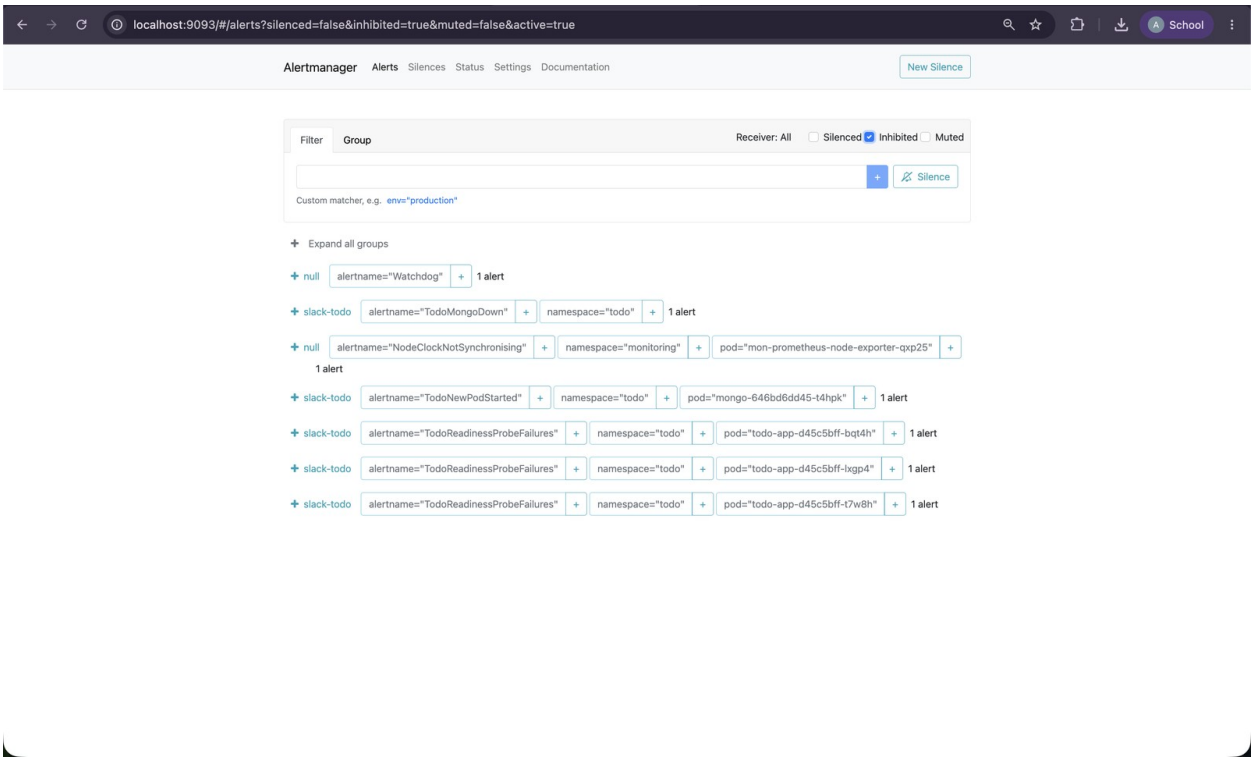


Figure 8.20: Inhibited ticked: the three TodoReadinessProbeFailures alerts exist but are suppressed by TodoMongoDown, which keep_firing_for holds active after Mongo returned.

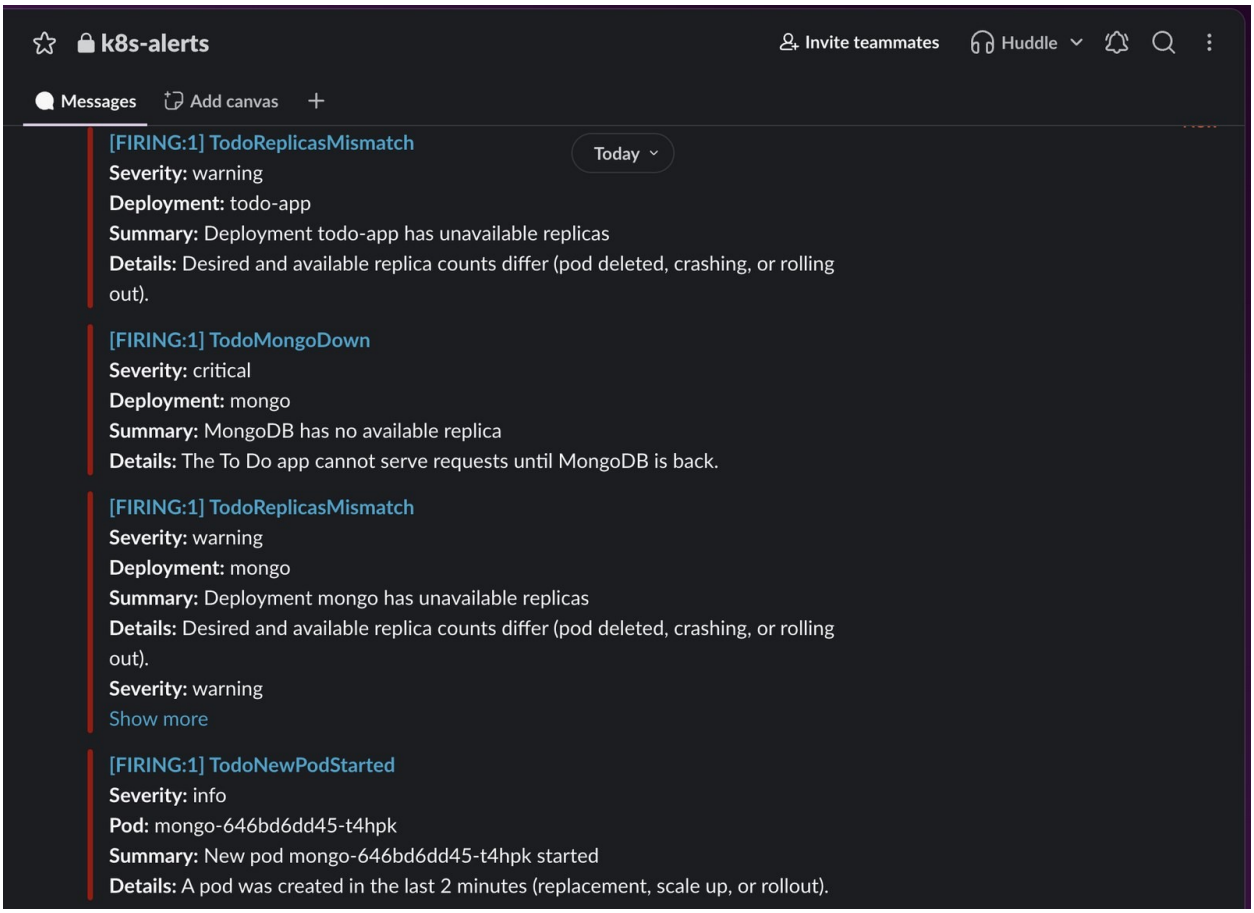


Figure 8.21: E3 in Slack: one critical root cause alert, TodoMongoDown (Deployment: mongo), plus replica mismatches. No readiness alerts.

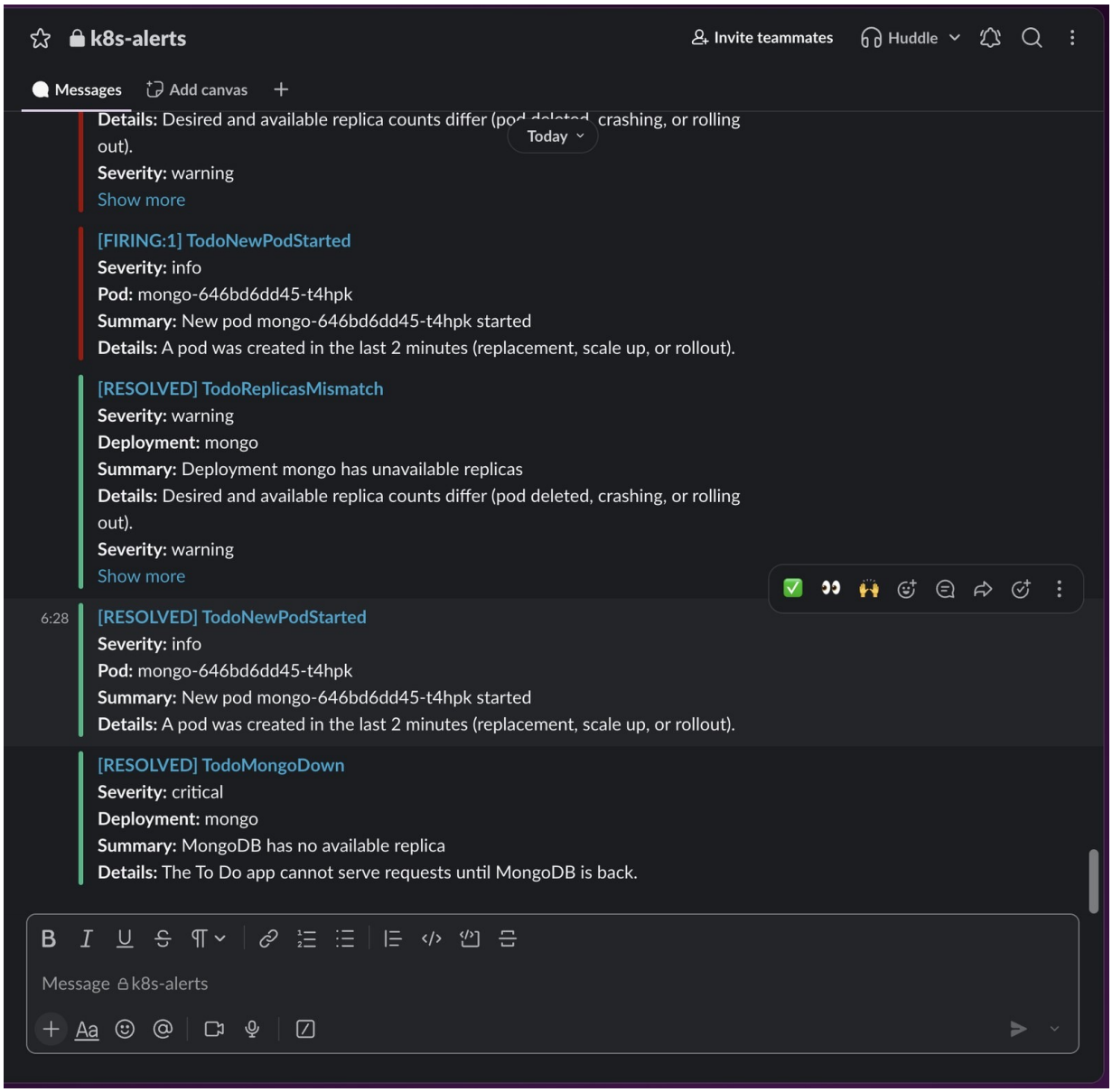


Figure 8.22: Everything resolves, TodoMongoDown last. No readiness alert was ever posted.

Edge Cases Handled

Risk	How we handled it
Database outage restarts every app pod	Liveness never touches Mongo; only readiness does (Part 7, T2).
A broken release takes the site down	maxUnavailable 0 and readiness probes keep old pods until new ones are ready; progressDeadlineSeconds 120 marks the rollout failed (Part 6).
kubectl set image drifts from Git	newTag in kustomization.yaml updated to v2 and committed; kubectl diff confirmed no drift.
Image fails on EKS (amd64) when built on an M1 Mac (arm64)	Every tag is a multi arch manifest with amd64 and arm64.
Overwritten tags leave stale cached images on nodes	Tags are never overwritten; fixes get new tags.
Secrets in Git	.env, the Mongo Secret and the Slack webhook live only in gitignored files or Kubernetes Secrets; a git grep scan returns only placeholders.
Two Mongo pods writing to one volume	Mongo uses the Recreate strategy and 1 replica.
Alert noise from chart alerts	Routed to the null receiver; only team=todo reaches Slack.
One outage, many alerts	Inhibition from TodoMongoDown to readiness alerts, with for: 1m and keep_firing_for: 3m so it covers the start and the end of the outage.
Alert shows the wrong pod	Deployment based rules aggregate with max by (namespace, deployment), so Slack shows the Deployment instead of the kube-state-metrics pod.
Short failures fall between scrapes	Rules use increase(...[2m]) and pod start time instead of instant values.
AWS free plan refuses t3.medium nodes	Replaced the node group with a free tier eligible type, m7i-flex.large, found with describe-instance-types.
EBS volume in the wrong zone for Mongo	gp3 StorageClass uses WaitForFirstConsumer, so the volume is created in the zone where Mongo is scheduled.
Orphaned AWS resources after teardown	Delete the Kubernetes objects first (ELB and EBS go with them), then the cluster.
Port 5000 taken on macOS	Compose publishes 8080; the container still listens on 5000.

Appendix: Commands in Run Order

Images (Part 2)

```
docker login -u thisisadityajha
docker buildx create --name multi --use && docker buildx inspect --bootstrap
docker build -t todo-flask:dev ./app
docker run --rm todo-flask:dev id
for v in v1 v2; do
  docker buildx build --builder multi --platform linux/amd64,linux/arm64 \
    --build-arg APP_VERSION=$v -t thisisadityajha/todo-flask:$v --push ./app
done
docker buildx imagetools inspect thisisadityajha/todo-flask:v1
```

Compose (Part 2)

```
docker compose up -d --build
docker compose ps
curl -s localhost:8080/healthz; curl -s localhost:8080/readyz; curl -s localhost:8080/version
docker compose down && docker compose up -d
docker compose stop mongo
curl -s -o /dev/null -w "readyz %{http_code}\n" localhost:8080/readyz
docker compose start mongo
docker compose logs web | grep -ci traceback
```

Minikube (Parts 3, 5, 7)

```
minikube start --driver=docker --cpus=4 --memory=8192
kubectl apply -f k8s/base/namespace.yaml
kubectl create secret generic mongo-credentials -n todo \
  --from-literal=MONGO_USER=<user> --from-literal=MONGO_PASSWORD=<password>
kubectl apply -k k8s/overlays/minikube
kubectl -n todo get pods -w
minikube service todo-app -n todo --url
for i in $(seq 1 9); do curl -s $URL/version; echo; done
kubectl -n todo delete pod <pod>
kubectl -n todo scale deployment todo-app --replicas=5
kubectl -n todo exec <pod> -- touch /tmp/unhealthy
kubectl -n todo scale deployment mongo --replicas=0
```

EKS (Part 4)

```
eksctl create cluster -f eks/cluster.yaml
aws eks update-kubeconfig --name todo-eks --region us-east-1
eksctl create addon -f eks/addon-ebs.yaml
kubectl apply -f k8s/base/namespace.yaml
kubectl create secret generic mongo-credentials -n todo \
  --from-literal=MONGO_USER=<user> --from-literal=MONGO_PASSWORD=<password>
kubectl apply -k k8s/overlays/eks
kubectl -n todo get all,pvc
for i in $(seq 1 12); do curl -s http://$ELB/version; echo; done
kubectl -n todo delete pod -l app=mongo
kubectl delete -k k8s/overlays/eks && eksctl delete cluster -f eks/cluster.yaml --wait
```

Rolling update (Part 6)

```
kubectl -n todo set image deploy/todo-app todo-app=thisisadityajha/todo-flask:v2
```

```
kubectl -n todo rollout status deploy/todo-app
kubectl -n todo rollout history deploy/todo-app
kubectl -n todo set image deploy/todo-app todo-app=thisisadityajha/todo-flask:v3-broken
kubectl -n todo rollout undo deploy/todo-app
kubectl -n todo set image deploy/todo-app todo-app=thisisadityajha/todo-flask:v9-does-not-
exist
kubectl -n todo rollout undo deploy/todo-app
kubectl apply -k k8s/overlays/minikube
```

Alerting (Part 8)

```
kubectl create namespace monitoring
kubectl create secret generic alertmanager-slack -n monitoring --from-literal=url="$SLACK_URL"
helm upgrade --install mon prometheus-community/kube-prometheus-stack -n monitoring \
  -f monitoring/values.yaml -f monitoring/alertmanager-slack.yaml
kubectl apply -f monitoring/todo-servicemonitor.yaml -f monitoring/todo-alert-rules.yaml
kubectl port-forward -n monitoring svc/mon-kube-prometheus-stack-prometheus 9090:9090
kubectl port-forward -n monitoring svc/mon-kube-prometheus-stack-alertmanager 9093:9093
curl -XPOST localhost:9093/api/v2/alerts -H 'Content-Type: application/json' \
  -d '[{"labels":{"alertname":"TodoManualTest","team":"todo","severity":"info"}}]'
kubectl -n todo delete pod <pod>                                          # E1
kubectl -n todo exec <pod> -- curl -s -XPOST localhost:5000/chaos/fail-liveness  # E2
kubectl -n todo scale deploy mongo --replicas=0; sleep 120; \
  kubectl -n todo scale deploy mongo --replicas=1                         # E3
```

Submission files

- app/Dockerfile and app/.dockerignore
- docker-compose.yml
- k8s/base (namespace, ConfigMap, Secret template, Mongo PVC, Deployment and Service, app Deployment and Service, kustomization), k8s/overlays/minikube, k8s/overlays/eks
- eks/cluster.yaml and eks/addon-ebs.yaml
- monitoring/values.yaml, todo-servicemonitor.yaml, todo-alert-rules.yaml, alertmanager-slack.yaml
- This report